

1 Production Performance Troubleshooting

When the phone rings and the answer must arrive in minutes, not hours.

1.1 A PostgreSQL database suddenly becomes slow. How will you troubleshoot it step by step?

Answer. Work outward from the database to the machine, never the reverse. Five minutes in this order finds the cause in almost every case:

STEP	CHECK	WHAT YOU ARE LOOKING FOR
1	Active sessions	How many, and what are they waiting on
2	Blocking	One stuck session holding hundreds behind it
3	Long transactions	<code>idle in transaction</code> holding the xmin horizon
4	Top queries	<code>pg_stat_statements</code> by total time
5	Machine	CPU, memory, disk I/O, disk free

```
-- 1. What is running right now, oldest first
SELECT pid, username, state, wait_event_type, wait_event,
       now() - query_start AS runtime,
       left(query, 90) AS query
FROM   pg_stat_activity
WHERE  state <> 'idle' AND backend_type = 'client backend'
ORDER BY query_start;

-- 2. Is anything blocked?
SELECT pid, pg_blocking_pids(pid) AS blocked_by, state, left(query,80)
FROM   pg_stat_activity
WHERE  cardinality(pg_blocking_pids(pid)) > 0;

-- 3. Oldest transaction still open
SELECT pid, username, state, now() - xact_start AS xact_age, left(query,80)
FROM   pg_stat_activity
WHERE  xact_start IS NOT NULL
ORDER BY xact_start
LIMIT 5;
```

THE SINGLE MOST USEFUL COLUMN

`wait_event_type`. If it reads `Lock` you have a blocking problem. `IO` means storage. `LWLock` means internal contention. `CPU` (a NULL wait event while active) means the queries themselves are expensive. That one column tells you which of the next four sections to read.

1.2 How do you identify the top 10 slow queries?

Answer. `pg_stat_statements`. Without it you are guessing. Enable it once and it costs almost nothing.

```
-- One-time setup (requires a restart, it is a shared_preload_library)
-- postgresql.conf:
--   shared_preload_libraries = 'pg_stat_statements'
--   pg_stat_statements.track = all
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;
```

```
-- Top 10 by TOTAL time - the queries actually costing you the most
SELECT round(total_exec_time::numeric, 0)      AS total_ms,
       calls,
       round(mean_exec_time::numeric, 2)      AS mean_ms,
       round(100 * total_exec_time / sum(total_exec_time) OVER (), 1) AS pct,
       rows,
       left(query, 100)                      AS query
FROM   pg_stat_statements
ORDER  BY total_exec_time DESC
LIMIT 10;

-- Top 10 by MEAN time - the queries that feel slow to a user
SELECT round(mean_exec_time::numeric,2) AS mean_ms, calls, left(query,100)
FROM   pg_stat_statements
WHERE  calls > 50
ORDER  BY mean_exec_time DESC
LIMIT 10;

-- Reset the counters before a test window
SELECT pg_stat_statements_reset();
```

TOTAL TIME, NOT MEAN TIME

A query averaging 5 ms but called 2 million times costs more than one taking 30 seconds once a day. Interviewers ask for “slow queries”; the professional answer sorts by `total_exec_time` first and explains why.

1.3 How do you use EXPLAIN and EXPLAIN ANALYZE?

Answer. `EXPLAIN` shows the plan the planner *intends* and executes nothing. `EXPLAIN ANALYZE` actually **runs the query** and shows what really happened.

```
-- Plan only, nothing executed - safe on any statement
EXPLAIN SELECT * FROM xxxx_orders WHERE customer_id = 42;

-- Runs the query. The form you should always use in practice:
EXPLAIN (ANALYZE, BUFFERS, VERBOSE, SETTINGS, FORMAT TEXT)
SELECT * FROM xxxx_orders WHERE customer_id = 42;
```

OPTION	WHAT IT ADDS
<code>ANALYZE</code>	Real row counts and real timings. Executes the query.
<code>BUFFERS</code>	Shared hit / read / dirtied blocks. Tells you cache versus disk.
<code>VERBOSE</code>	Output column list and schema-qualified names.
<code>SETTINGS</code>	Any non-default planner setting in force. Explains “works on my machine”.
<code>WAL</code>	WAL generated. Useful on <code>INSERT</code> / <code>UPDATE</code> plans.

EXPLAIN ANALYZE ON A WRITE STATEMENT REALLY WRITES

Wrap it in a transaction you roll back:

```
BEGIN;
EXPLAIN (ANALYZE, BUFFERS) DELETE FROM xxxx_orders WHERE status = 'CANCELLED';
ROLLBACK;
```

Reading the output

```
Seq Scan on xxxx_orders (cost=0.00..18334.00 rows=1000 width=64)
      (actual time=0.015..142.331 rows=998338 loops=1)
    Filter: (customer_id = 42)
    Rows Removed by Filter: 1661662
    Buffers: shared hit=120 read=8214
    Planning Time: 0.184 ms
    Execution Time: 268.442 ms
```

- **rows=1000 estimated versus rows=998338 actual** — a 1000× misestimate. This is the number one thing to look for. It means statistics are stale: run `ANALYZE`.
- **Rows Removed by Filter** high — the query read 1.6 million rows to discard them. A missing index.
- **read=8214** — blocks fetched from disk rather than cache.
- **Planning Time** unusually high — too many partitions or too many joins.

1.4 What is the difference between Seq Scan, Index Scan and Bitmap Heap Scan?

NODE	HOW IT WORKS	CHOSEN WHEN
Seq Scan	Reads every block of the table in physical order	The table is small, or the query returns a large fraction of it. Often the correct choice — a sequential scan is not automatically a bug.
Index Scan	Walks the index, then fetches each matching row from the heap, one at a time, in index order	Few rows match, and they are physically clustered. Random I/O per row, so it loses on large result sets.
Index Only Scan	Answers entirely from the index without touching the heap	Every column needed is in the index and the visibility map says the pages are all-visible. The fastest node there is.
Bitmap Heap Scan	Builds a bitmap of matching block numbers from the index, sorts it, then reads the heap in physical order	A medium number of rows match. Converts random I/O into sequential I/O. Frequently combines several indexes via <code>BitmapAnd</code> / <code>BitmapOr</code> .

Few rows 	Medium number 	Most rows
Index Scan ----->	Bitmap Heap Scan ----->	Seq Scan
(random I/O, one row at a time)	(index -> bitmap -> sorted heap reads)	(pure sequential, no index overhead)

A RED FLAG WORTH KNOWING

Recheck `Cond` together with `Heap Blocks: lossy=N` in a Bitmap Heap Scan means the bitmap outgrew `work_mem` and degraded from per-row to per-block precision. Raising `work_mem` for that query often fixes it outright.

1.5 How do you identify whether an index is being used?

```
-- Usage counters per index. idx_scan = 0 means never used since stats reset.
SELECT s.schemaname, s.relname AS table_name, s.indexrelname AS index_name,
       s.idx_scan, s.idx_tup_read, s.idx_tup_fetch,
       pg_size_pretty(pg_relation_size(s.indexrelid)) AS index_size
FROM   pg_stat_user_indexes s
JOIN   pg_index i ON i.indexrelid = s.indexrelid
WHERE  s.schemaname NOT IN ('pg_catalog','information_schema')
ORDER  BY s.idx_scan, pg_relation_size(s.indexrelid) DESC;
```

```
-- Unused indexes that are costing you write performance and disk
SELECT schemaname, relname, indexrelname,
       pg_size_pretty(pg_relation_size(indexrelid)) AS wasted
FROM   pg_stat_user_indexes
WHERE  idx_scan = 0
      AND indexrelid NOT IN (SELECT conindid FROM pg_constraint WHERE contype IN ('p','u'))
ORDER  BY pg_relation_size(indexrelid) DESC;
```

```
-- Sequential scans on large tables - the inverse signal
SELECT relname, seq_scan, seq_tup_read, idx_scan,
       pg_size_pretty(pg_relation_size(relid)) AS size
FROM   pg_stat_user_tables
WHERE  seq_scan > 0 AND pg_relation_size(relid) > 100*1024*1024
ORDER  BY seq_tup_read DESC;
```

TWO CAUTIONS BEFORE YOU DROP ANYTHING

Counters reset on `pg_stat_reset()` and on a crash, so `idx_scan = 0` may only mean “not used since Tuesday”. And an index enforcing a primary key or unique constraint may show zero scans while still being essential. Confirm the counter has a long baseline, and exclude constraint-backing indexes as the query above does.

1.6 A query was fast yesterday but is slow today. What would you check?

Answer. The query did not change, so something around it did. Check in this order — the first three cover the overwhelming majority.

#	CAUSE	HOW TO CONFIRM
1	Stale statistics — the table grew or its distribution shifted	<code>last_analyze</code> in <code>pg_stat_user_tables</code> ; estimated versus actual rows in the plan
2	Plan flip — the planner chose a different plan	Compare <code>EXPLAIN</code> now against the known-good plan
3	Generic plan — a prepared statement switched off custom plans after five executions	<code>SET plan_cache_mode = force_custom_plan;</code> and re-test
4	Bloat — dead tuples inflated the table or index	<code>n_dead_tup</code> , and the bloat query in section 3
5	Blocking — it is not slow, it is waiting	<code>pg_blocking_pids()</code>
6	Data volume — it genuinely has more rows to process	<code>n_live_tup</code> trend
7	Index dropped or invalid	<code>SELECT * FROM pg_index WHERE NOT indisvalid;</code>
8	Cache cold — after a restart or failover	<code>BUFFERS</code> showing high <code>read</code> , low <code>hit</code>

```
-- Statistics currency, the first thing to rule out
SELECT relname, n_live_tup, n_dead_tup, last_analyze, last_autoanalyze
FROM   pg_stat_user_tables
WHERE  relname = 'xxxx_orders';

ANALYZE xxxx_orders;  -- then re-run EXPLAIN and compare

-- Invalid indexes (a failed CREATE INDEX CONCURRENTLY leaves these behind)
SELECT indexrelid::regclass AS index_name, indrelid::regclass AS table_name
FROM   pg_index WHERE NOT indisvalid;
```

1.7 How do you troubleshoot high CPU usage?

Answer. High CPU in PostgreSQL is almost always a small number of expensive queries, not the server being “overloaded”. Find them, do not tune blindly.

```
-- Step 1: which backends are burning CPU (active, not waiting on anything)
SELECT pid, username, now() - query_start AS runtime,
       state, wait_event_type, wait_event, left(query, 100)
FROM   pg_stat_activity
WHERE  state = 'active' AND wait_event IS NULL
ORDER  BY query_start;
```

A backend that is `active` with a `NULL` `wait_event` is on the CPU right now. That is your list.

```
-- Step 2: correlate with the OS (Linux)
top -c -o %CPU
ps -eo pid,pcpu,etime,cmd --sort=-pcpu | head -20
```

The PostgreSQL backend process ID in `ps` is the same number as `pg_stat_activity.pid`, so you can map an OS process straight to a query.

```
-- Step 3: total CPU consumers over time
SELECT round(total_exec_time::numeric) AS total_ms, calls,
       round(mean_exec_time::numeric,2) AS mean_ms, left(query,90)
FROM   pg_stat_statements ORDER BY total_exec_time DESC LIMIT 10;
```

Common causes, in order of frequency

- **Missing index** forcing repeated sequential scans on a large table.
- **Bad plan from stale statistics** — nested loop over millions of rows instead of a hash join.
- **Excessive parallelism** — `max_parallel_workers_per_gather` too high, so one query occupies many cores.
- **Application retry storm** — the same failing query resubmitted in a loop. Look for high `calls` with low `rows`.
- **Autovacuum falling behind** then running many workers at once.

1.8 How do you troubleshoot high memory usage?

Answer. Separate what is shared once from what multiplies per connection. Nearly all runaway memory is `work_mem` multiplied by concurrency.

MEMORY AREA	SCOPE	SIZING NOTE
<code>shared_buffers</code>	Shared, once	~25% of RAM. Allocated at startup.
Lock table (<code>max_locks_per_transaction</code>)	Shared, once	~270 bytes per slot. Allocated at startup.
WAL buffers	Shared, once	Usually <code>-1</code> (auto, 1/32 of <code>shared_buffers</code>).
<code>work_mem</code>	Per sort / hash / node	This is the dangerous one. One query with 3 sorts and 4 parallel workers can use it 12 times.
<code>maintenance_work_mem</code>	Per maintenance operation	Used by VACUUM, CREATE INDEX, ALTER TABLE.
<code>temp_buffers</code>	Per connection	For temporary tables.
Backend + relcache	Per connection	1–3 MB base, plus cached metadata for every relation the session has touched.

THE FORMULA THAT CAUSES MOST OOM INCIDENTS

$\text{worst case} \approx \text{shared_buffers} + (\text{work_mem} \times \text{nodes} \times \text{parallel_workers} \times \text{active_connections})$

With `work_mem = 256MB` and 100 active connections each running a two-sort query, that is 51 GB of potential demand. Keep `work_mem` modest globally and raise it per session for the few reports that need it.

```
-- Per-session raise, not a global change
SET work_mem = '256MB';
SELECT ... big report ... ;
RESET work_mem;

-- Check what temp files are being spilled (work_mem too small)
SELECT datname, temp_files, pg_size_pretty(temp_bytes) AS temp_written
FROM   pg_stat_database WHERE temp_files > 0 ORDER BY temp_bytes DESC;

-- Log every spill over 10 MB so you can find them later
ALTER SYSTEM SET log_temp_files = '10MB';
SELECT pg_reload_conf();
```

1.9 How do you identify a query consuming excessive I/O?

```
-- PostgreSQL 16 and later: per-backend-type I/O statistics
SELECT backend_type, object, context,
       reads, writes, extends, hits, evictions,
       round(read_time::numeric,0) AS read_ms
FROM   pg_stat_io
WHERE  reads > 0 OR writes > 0
ORDER BY reads DESC;
```

```
-- All versions: queries by block reads, if track_io_timing is on
-- ALTER SYSTEM SET track_io_timing = on; SELECT pg_reload_conf();
SELECT round(total_exec_time::numeric) AS total_ms,
       shared_blks_read AS blocks_from_disk,
       shared_blks_hit AS blocks_from_cache,
       round(blk_read_time::numeric) AS io_wait_ms,
       calls, left(query, 80)
FROM   pg_stat_statements
ORDER BY shared_blks_read DESC
LIMIT 10;
```

```
-- Cache hit ratio per table. Below ~0.95 on a hot table warrants attention.
SELECT relname,
       heap_blks_hit, heap_blks_read,
       round(heap_blks_hit::numeric / NULLIF(heap_blks_hit + heap_blks_read,0), 4) AS hit_ratio
FROM   pg_statio_user_tables
WHERE  heap_blks_read > 0
ORDER BY heap_blks_read DESC LIMIT 15;
```

```
-- Sessions waiting on I/O right now
SELECT pid, wait_event_type, wait_event, left(query,80)
FROM   pg_stat_activity WHERE wait_event_type = 'IO';

-- OS level
iostat -xz 1 5
iotop -oPa
```

1.10 What PostgreSQL parameters would you check for performance tuning?

PARAMETER	TYPICAL STARTING POINT	WHY
<code>shared_buffers</code>	25% of RAM	PostgreSQL's own page cache. Beyond ~40% you fight the OS cache.
<code>effective_cache_size</code>	50–75% of RAM	A hint to the planner only — allocates nothing. Too low pushes it toward sequential scans.
<code>work_mem</code>	4–64 MB	Per sort or hash node. Multiplies. Raise per session, not globally.
<code>maintenance_work_mem</code>	512 MB – 2 GB	Speeds up VACUUM and index builds dramatically.
<code>random_page_cost</code>	1.1 on SSD (default 4.0)	The default assumes spinning disks. On SSD this single change is often the biggest win available.
<code>effective_io_concurrency</code>	200 on SSD/NVMe	Enables deeper prefetch for bitmap scans.
<code>max_wal_size</code>	4–16 GB	Too small causes constant checkpoints and I/O spikes.
<code>checkpoint_completion_target</code>	0.9	Spreads checkpoint writes instead of bursting.
<code>max_connections</code>	100–300 + a pooler	PostgreSQL forks a process per connection. Use PgBouncer beyond this.
<code>max_locks_per_transaction</code>	64, raise for partitioned schemas	Sizes the shared lock table. Critical with many partitions.
<code>autovacuum_vacuum_scale_factor</code>	0.05 or lower on big tables	Default 0.2 means a billion-row table waits for 200 million dead rows.
<code>track_io_timing</code>	on	Without it you cannot attribute I/O wait to queries.
<code>log_min_duration_statement</code>	1000 (ms)	Logs anything over a second. Your cheapest diagnostic.
<code>log_lock_waits</code>	on	Records who blocked whom, after <code>deadlock_timeout</code> .

```
-- See everything set away from default on this instance
SELECT name, setting, unit, source, boot_val
FROM   pg_settings
WHERE  source NOT IN ('default','override')
ORDER BY name;
```

2 Locking and Blocking

The most common production incident, and the one most often misdiagnosed as “the database is slow”.

2.1 How do you find blocking sessions?

Answer. Use `pg_blocking_pids()`. It is available from 9.6 onward and replaces the old self-join on `pg_locks` that people still copy from blog posts.

```
-- The one query to remember
SELECT  blocked.pid           AS blocked_pid,
        blocked.username      AS blocked_user,
        blocked.wait_event_type,
        blocked.wait_event,
        now() - blocked.query_start AS blocked_for,
        left(blocked.query, 70)   AS blocked_query,
        blocking.pid          AS blocking_pid,
        blocking.username      AS blocking_user,
        blocking.state         AS blocking_state,
        now() - blocking.xact_start AS blocking_xact_age,
        left(blocking.query, 70)  AS blocking_query
FROM    pg_stat_activity blocked
JOIN    LATERAL unnest(pg_blocking_pids(blocked.pid)) AS bp(pid) ON true
JOIN    pg_stat_activity blocking ON blocking.pid = bp.pid
WHERE   cardinality(pg_blocking_pids(blocked.pid)) > 0
ORDER   BY blocked_for DESC;
```

```
-- Find the root blocker in a chain: it is blocked by nobody
SELECT pid, username, state, now() - xact_start AS xact_age, left(query, 90)
FROM    pg_stat_activity
WHERE   cardinality(pg_blocking_pids(pid)) = 0
        AND pid IN (SELECT unnest(pg_blocking_pids(pid)) FROM pg_stat_activity);
```

BLOCKING CHAINS

A blocks B, B blocks C, C blocks forty sessions. Killing C or B changes nothing. Always find the **root** — the session that is blocking others while being blocked by none. That second query does exactly that.

2.2 How do you identify which query is holding a lock?

```
-- Every lock currently held or awaited, with the owning query
SELECT l.pid, l.locktype, l.mode, l.granted,
       COALESCE(c.relname, l.locktype) AS object,
       a.username, a.state,
       now() - a.xact_start AS xact_age,
       left(a.query, 80) AS query
FROM    pg_locks l
LEFT JOIN pg_class c ON c.oid = l.relation
JOIN    pg_stat_activity a ON a.pid = l.pid
WHERE   NOT l.granted OR l.mode LIKE '%Exclusive%'
ORDER   BY l.granted, xact_age DESC;
```

```
-- Who holds a lock on one specific table
SELECT a.pid, a.username, l.mode, l.granted, a.state, left(a.query, 90)
FROM    pg_locks l
JOIN    pg_stat_activity a ON a.pid = l.pid
WHERE   l.relation = 'xxxx_orders'::regclass
ORDER   BY l.granted;
```

Lock modes you must recognise

MODE	TAKEN BY	CONFLICTS WITH
ACCESS SHARE	SELECT	Only ACCESS EXCLUSIVE
ROW SHARE	SELECT FOR UPDATE	EXCLUSIVE, ACCESS EXCLUSIVE
ROW EXCLUSIVE	INSERT , UPDATE , DELETE	SHARE and above
SHARE UPDATE EXCLUSIVE	VACUUM , ANALYZE , CREATE INDEX CONCURRENTLY	Itself and above
SHARE	CREATE INDEX (non-concurrent)	Writes
ACCESS EXCLUSIVE	ALTER TABLE , DROP , TRUNCATE , VACUUM FULL , REINDEX	Everything, including SELECT

THE CLASSIC PRODUCTION OUTAGE

An `ALTER TABLE` queues for an `ACCESS EXCLUSIVE` lock behind one long-running `SELECT` . While it waits, **every subsequent query queues behind it** — including plain `SELECT`s that would otherwise be compatible. The table appears completely frozen. Always run DDL with a lock timeout:

```
SET lock_timeout = '5s';
ALTER TABLE xxxx_orders ADD COLUMN note text;
```

2.3 How do you terminate a blocking session?

```
-- 1. Try cancelling the query first (polite - the transaction survives)
SELECT pg_cancel_backend(12345);

-- 2. If that does not work, terminate the connection
SELECT pg_terminate_backend(12345);

-- 3. Terminate everything idle in transaction for more than 30 minutes
SELECT pid, username, now() - state_change AS idle_for,
       pg_terminate_backend(pid)
FROM   pg_stat_activity
WHERE  state = 'idle in transaction'
       AND now() - state_change > interval '30 minutes'
       AND pid <> pg_backend_pid();

-- 4. Disconnect everyone from one database (before a restore, for example)
SELECT pg_terminate_backend(pid)
FROM   pg_stat_activity
WHERE  datname = 'xxxx' AND pid <> pg_backend_pid();
```

ALWAYS EXCLUDE YOURSELF

`AND pid <> pg_backend_pid()` is not optional. Without it you terminate your own session part-way through and the statement stops. Also: run a `SELECT` of the list first, read it, then add the terminate. Never fire a bulk terminate you have not eyeballed.

2.4 What is the difference between `pg_cancel_backend()` and `pg_terminate_backend()`?

	PG_CANCEL_BACKEND(PID)	PG_TERMINATE_BACKEND(PID)
Effect	Cancels the currently running statement	Terminates the entire backend process

	PG_CANCEL_BACKEND(PID)	PG_TERMINATE_BACKEND(PID)
Signal	SIGINT	SIGTERM
Connection	Stays open	Closed
Transaction	Survives, in aborted state until rollback	Rolled back
Client sees	ERROR: canceling statement due to user request	FATAL: terminating connection due to administrator command
Works on idle sessions	No — there is no statement to cancel	Yes
Use it for	A runaway query on a session you want to keep	idle in transaction, or when cancel had no effect

Rule: always try `pg_cancel_backend()` first. Escalate to `pg_terminate_backend()` only if the query does not respond — some operations only check for cancellation at intervals. Both return `true` if the signal was sent, which is **not** the same as the session having gone; re-query `pg_stat_activity` to confirm.

```
-- PostgreSQL 14+: wait for it and get a real answer
SELECT pg_terminate_backend(12345, timeout => 5000); -- milliseconds
```

2.5 What causes deadlocks and how do you troubleshoot them?

Answer. A deadlock is two or more transactions each holding a lock the other needs. PostgreSQL detects the cycle automatically after `deadlock_timeout` (default 1 second) and **kills one transaction** so the rest proceed. The application receives SQLSTATE `40P01`.

```
Session A                               Session B
-----                               -
BEGIN;                                BEGIN;
UPDATE accounts SET .. WHERE id=1;    UPDATE accounts SET .. WHERE id=2;
(holds lock on row 1)                (holds lock on row 2)

UPDATE accounts SET .. WHERE id=2;    UPDATE accounts SET .. WHERE id=1;
(waits for B)                        (waits for A)
      ^                               ^
      +---- DEADLOCK -----+
PostgreSQL detects the cycle and aborts one of them.
```

Troubleshooting

```
-- Make sure deadlocks are logged in full
ALTER SYSTEM SET log_lock_waits = on;
ALTER SYSTEM SET deadlock_timeout = '1s';
SELECT pg_reload_conf();

-- Count deadlocks per database - are they rising?
SELECT datname, deadlocks, conflicts, temp_files
FROM   pg_stat_database WHERE datname = 'xxxx';
```

The server log gives you both sides of the cycle, which is what you actually need:

```
ERROR:  deadlock detected
DETAIL:  Process 4521 waits for ShareLock on transaction 89234; blocked by process 4498.
        Process 4498 waits for ShareLock on transaction 89235; blocked by process 4521.
        Process 4521: UPDATE xxxx_accounts SET balance = ... WHERE id = 2;
        Process 4498: UPDATE xxxx_accounts SET balance = ... WHERE id = 1;
HINT:   See server log for query details.
```

Prevention

- **Consistent ordering.** Always touch rows and tables in the same order in every code path — ascending primary key, for instance. This eliminates most deadlocks outright.
- **Shorter transactions.** Do not hold locks across application logic, user think-time or network calls.
- **Take the strongest lock first** rather than upgrading a share lock to an exclusive one mid-transaction.
- **Retry logic.** A deadlock is a normal, recoverable condition. Catch `40P01` and retry the transaction once or twice with backoff.

2.6 What happens if a transaction remains open for a long time?

Answer. Four separate harms, and the third is the one that silently destroys a database.

CONSEQUENCE	DETAIL
Locks are held	Every lock taken stays until commit or rollback. DDL queues behind it, and everything queues behind the DDL.
VACUUM is crippled	An open transaction holds back the xmin horizon . VACUUM cannot remove any dead tuple newer than the oldest running transaction — anywhere in the cluster , in any database.
Bloat grows unchecked	Dead tuples accumulate because they cannot be reclaimed. Tables and indexes swell; queries slow down for reasons that appear unrelated.
Wraparound risk	If it persists long enough, the cluster approaches transaction ID wraparound and will eventually refuse new transactions.

```
-- The xmin horizon - what is holding VACUUM back cluster-wide
SELECT pid, username, datname, state,
       now() - xact_start AS xact_age,
       backend_xmin,
       left(query, 70)
FROM   pg_stat_activity
WHERE  backend_xmin IS NOT NULL
ORDER BY age(backend_xmin) DESC
LIMIT 10;
```

```
-- Prevention: set these and stop having the problem
ALTER SYSTEM SET idle_in_transaction_session_timeout = '10min';
ALTER SYSTEM SET statement_timeout = '5min';           -- careful on OLAP systems
ALTER SYSTEM SET lock_timeout = '10s';
SELECT pg_reload_conf();
```

REPLICATION SLOTS AND PREPARED TRANSACTIONS DO THE SAME THING

An abandoned replication slot or an orphaned two-phase `PREPARE` also pins the xmin horizon — with no session visible in `pg_stat_activity` to explain it. Check `pg_replication_slots` and `pg_prepared_xacts` whenever bloat grows and no long transaction is visible.

2.7 How do you identify idle in transaction sessions?

```
-- Idle in transaction, worst first
SELECT pid, username, datname, client_addr,
       state,
       now() - state_change AS idle_duration,
       now() - xact_start AS transaction_age,
       left(query, 80) AS last_statement
FROM   pg_stat_activity
WHERE  state IN ('idle in transaction', 'idle in transaction (aborted)')
ORDER BY xact_start;
```

```
-- Summary by state - a quick health picture
SELECT state, count(*), max(now() - state_change) AS longest
FROM   pg_stat_activity
WHERE  backend_type = 'client backend'
GROUP BY state ORDER BY count(*) DESC;
```

What the states mean:

- `idle` — connected, no transaction open. Harmless.
- `idle in transaction` — **a transaction is open and doing nothing**. Holding locks and the xmin horizon. This is the harmful one.
- `idle in transaction (aborted)` — the transaction errored and the application has not sent `ROLLBACK`. Still holds the snapshot.
- `active` — executing.

ALMOST ALWAYS AN APPLICATION BUG

It means code opened a transaction and then did something slow that is not database work — an HTTP call, a file write, waiting for user input — or an ORM did not close a unit of work on an error path. `idle_in_transaction_session_timeout` is the safety net, not the fix.

3 VACUUM and Autovacuum

The topic that separates a PostgreSQL DBA from someone who has used PostgreSQL.

3.1 What is VACUUM?

Answer. PostgreSQL uses MVCC: an `UPDATE` does not overwrite a row, it writes a **new version** and marks the old one dead. A `DELETE` only marks dead. Nothing is physically removed. `VACUUM` is the process that reclaims those dead tuples.

```
UPDATE xxxx_orders SET status='SHIPPED' WHERE id=100;
```

BEFORE		AFTER	
+-----+		+-----+	
id=100 PENDING	-->	id=100 PENDING (DEAD)	<-- still on disk
+-----+		+-----+	
		id=100 SHIPPED (LIVE)	<-- new version
		+-----+	

VACUUM marks the dead tuple's space REUSABLE (not returned to the OS).

What VACUUM does:

- Marks space occupied by dead tuples as **reusable by that table**.
- Updates the **visibility map**, which is what makes Index Only Scans possible.
- **Freezes** old transaction IDs, preventing wraparound.
- Updates the free space map.
- `VACUUM ANALYZE` additionally refreshes planner statistics.

```
VACUUM xxxx_orders;           -- reclaim, no lock conflict with reads/writes
VACUUM (VERBOSE, ANALYZE) xxxx_orders; -- with output and fresh statistics
VACUUM (PARALLEL 4) xxxx_orders;  -- parallel index vacuuming (PG 13+)
```

3.2 What is the difference between VACUUM and VACUUM FULL?

	VACUUM	VACUUM FULL
Lock taken	<code>SHARE UPDATE EXCLUSIVE</code> — reads and writes continue	<code>ACCESS EXCLUSIVE</code> — the table is completely unavailable
Mechanism	Marks dead space reusable in place	Rewrites the entire table into a new file, then swaps
Space returned to OS	No (except empty trailing pages)	Yes
Extra disk needed	None	Roughly the full size of the table again
Indexes	Cleaned in place	Rebuilt from scratch
Speed	Fast, incremental	Slow, proportional to table size
Safe in production	Yes — runs automatically all day	No — maintenance window only

WHY VACUUM FULL IS DANGEROUS IN PRODUCTION

It takes an `ACCESS EXCLUSIVE` lock for its entire duration. On a 500 GB table that can be hours during which **every query against that table blocks, including SELECT**. It also needs another 500 GB free — run it on a nearly full disk and you take the database down. Prefer `pg_repack`, which achieves the same result online.

```
-- The safe alternative: rebuild online, no exclusive lock for the duration
pg_repack -d xxxx -t public.xxxx_orders --jobs 4

-- Rebuild indexes online (PG 12+)
REINDEX TABLE CONCURRENTLY xxxx_orders;
```

3.3 What is autovacuum, and why is it important?

Answer. Autovacuum is a background daemon that launches VACUUM and ANALYZE automatically when a table crosses a change threshold. It is the mechanism that keeps a PostgreSQL database healthy without human intervention.

The trigger formula:

```
vacuum threshold = autovacuum_vacuum_threshold + autovacuum_vacuum_scale_factor * reltuples
                  = 50 + 0.2 * reltuples          (defaults)

analyze threshold = autovacuum_analyze_threshold + autovacuum_analyze_scale_factor * reltuples
                   = 50 + 0.1 * reltuples          (defaults)
```

THE DEFAULT THAT BREAKS LARGE TABLES

`scale_factor = 0.2` means a table must accumulate **20% dead rows** before autovacuum touches it. On a 500 million row table that is **100 million dead tuples** — hundreds of gigabytes of bloat before anything happens. Override it per table on anything large:

```
ALTER TABLE xxxx_orders SET (
    autovacuum_vacuum_scale_factor = 0.01,    -- 1% instead of 20%
    autovacuum_vacuum_threshold   = 1000,
    autovacuum_analyze_scale_factor = 0.005,
    autovacuum_vacuum_cost_limit  = 2000      -- let it work faster
);
```

Why it matters: without it, dead tuples accumulate forever, tables and indexes bloat, queries slow down, disk fills, statistics go stale so plans degrade, and eventually transaction IDs approach wraparound and the cluster stops accepting writes. Disabling autovacuum is never the answer to a performance problem — it is how you create a much larger one.

```
-- Is autovacuum running right now, and on what?
SELECT pid, datname, relid::regclass AS table, phase,
       heap_blks_total, heap_blks_scanned, heap_blks_vacuumed,
       round(100.0 * heap_blks_scanned / NULLIF(heap_blks_total,0), 1) AS pct
FROM   pg_stat_progress_vacuum;
```

3.4 How do you identify tables suffering from table or index bloat?

```
-- Quick signal: dead tuple ratio per table
SELECT schemaname, relname,
       n_live_tup, n_dead_tup,
       round(100.0 * n_dead_tup / NULLIF(n_live_tup + n_dead_tup, 0), 2) AS dead_pct,
       last_vacuum, last_autovacuum, last_analyze, last_autoanalyze,
       pg_size_pretty(pg_total_relation_size(relid)) AS total_size
FROM   pg_stat_user_tables
WHERE  n_dead_tup > 1000
ORDER  BY n_dead_tup DESC
LIMIT  20;
```

```
-- Precise measurement (scans the table, so use on one table at a time)
CREATE EXTENSION IF NOT EXISTS pgstattuple;

SELECT * FROM pgstattuple('xxxx_orders');
-- table_len | tuple_count | tuple_percent | dead_tuple_count | dead_tuple_percent | free_percent

SELECT * FROM pgstatindex('idx_xxxx_orders_customer');
-- leaf_fragmentation and avg_leaf_density tell you index health
```

```
-- Cheap estimate across the whole database, no table scan
SELECT current_database(), schemaname, tblname,
       pg_size_pretty(bs*tblpages::bigint) AS real_size,
       pg_size_pretty((tblpages-est_tblpages_ff)*bs::bigint) AS bloat_size,
       round(100 * (tblpages - est_tblpages_ff)/tblpages::numeric, 1) AS bloat_pct
FROM ( /* use the standard pgsql-bloat-estimation query here */ ) x
WHERE tblpages > 128;
```

Rules of thumb: under 20% bloat is normal and healthy — that free space gets reused. Above 40% on a large, actively queried table is worth acting on. Above 100% (the table is more than twice its useful size) means autovacuum has not been keeping up and you should find out why before repacking.

3.5 Why does a table continue consuming disk space after deleting millions of rows?

Answer. Because `DELETE` does not free anything. It marks tuples dead. `VACUUM` then marks that space **reusable by the same table** — it does not hand it back to the operating system. The file on disk stays the same size, and the table refills into that space over time.

DELETE 5,000,000 rows	-> file size unchanged, 5M dead tuples
VACUUM	-> file size unchanged, space marked REUSABLE
VACUUM FULL / pg_repack	-> file rewritten, space RETURNED to OS

To actually reclaim the disk:

METHOD	LOCKS TABLE	WHEN TO USE
<code>VACUUM FULL</code>	Yes, exclusive	Maintenance window only. Needs 2× disk.
<code>pg_repack</code>	Briefly only	Production. Online rebuild. Needs 2× disk.
<code>CLUSTER</code>	Yes, exclusive	When you also want physical ordering by an index.
<code>TRUNCATE</code>	Yes, exclusive	Only if removing all rows. Instant, frees everything.
Drop the partition	Brief	The best answer if the table is partitioned by the delete key.

THE DESIGN LESSON

If you routinely delete large date ranges, partition the table by date. `DROP TABLE xxxx_orders_2024_01` is instantaneous, returns all the space, and needs no `VACUUM`. That is the difference between a design that ages well and one that needs a maintenance window every quarter.

3.6 What is transaction ID wraparound?

Answer. PostgreSQL transaction IDs are **32-bit** — about 4 billion values, of which roughly 2 billion can be “in the past” at any moment. Visibility is decided by comparing XIDs, so if the counter wrapped around, transactions from the past would suddenly appear to be in the future and **data would silently vanish**.

PostgreSQL prevents this by **freezing**: `VACUUM` marks sufficiently old rows with a special frozen marker meaning “visible to everyone, forever”. Wraparound only threatens a database where `VACUUM` has not run for a very long time.

```
-- How close is each database to the limit?
SELECT datname,
       age(datfrozenxid) AS xid_age,
       2000000000 - age(datfrozenxid) AS xids_remaining,
       round(100.0 * age(datfrozenxid) / 2000000000, 1) AS pct_to_wraparound
FROM   pg_database
ORDER  BY age(datfrozenxid) DESC;

-- Which tables are the oldest?
SELECT c.relname, age(c.relfrozenxid) AS xid_age,
       pg_size_pretty(pg_total_relation_size(c.oid)) AS size
FROM   pg_class c
JOIN   pg_namespace n ON n.oid = c.relnamespace
WHERE  c.relkind IN ('r','m','t') AND n.nspname NOT LIKE 'pg_%'
ORDER  BY age(c.relfrozenxid) DESC
LIMIT  20;
```

XID AGE	WHAT HAPPENS	ACTION
200 million	Anti-wraparound autovacuum triggers (<code>autovacuum_freeze_max_age</code>) — it runs even if autovacuum is disabled	Normal. Let it run.
~1.9 billion	<code>WARNING: database "xxxx" must be vacuumed within N transactions</code>	Act now.
~2 billion	The database stops accepting new transactions. Read-only, effectively down.	Single-user mode recovery.

NEVER KILL AN ANTI-WRAPAROUND AUTOVACUUM

It looks like a runaway process holding a lock and people terminate it. It will restart immediately, having made no progress, and the clock keeps ticking. Let it finish. If it must be faster, raise `autovacuum_vacuum_cost_limit` and lower `autovacuum_vacuum_cost_delay` instead.

3.7 How do you troubleshoot an autovacuum problem?

```
-- 1. Is autovacuum even on?
SHOW autovacuum;           -- must be 'on'
SHOW track_counts;        -- must be 'on', autovacuum depends on it
SHOW autovacuum_max_workers; -- default 3
SHOW autovacuum_naptime;   -- default 1min

-- 2. Which tables are overdue?
SELECT relname, n_live_tup, n_dead_tup,
       last_autovacuum, last_autoanalyze,
       autovacuum_count, analyze_count
FROM   pg_stat_user_tables
WHERE  n_dead_tup > 10000
ORDER BY n_dead_tup DESC LIMIT 20;

-- 3. What is autovacuum doing at this moment?
SELECT p.pid, p.datname, p.relid::regclass, p.phase,
       round(100.0*p.heap_blks_scanned/NULLIF(p.heap_blks_total,0),1) AS pct,
       now() - a.query_start AS running_for
FROM   pg_stat_progress_vacuum p
JOIN   pg_stat_activity a ON a.pid = p.pid;

-- 4. Is something pinning the xmin horizon so VACUUM cannot reclaim?
SELECT slot_name, active, wal_status, pg_size_pretty(
       pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)) AS retained_wal
FROM   pg_replication_slots;
SELECT * FROM pg_prepared_xacts;
SELECT max(age(backend_xmin)) FROM pg_stat_activity;
```

The five causes, in order of frequency

1. **Long-running transaction or abandoned replication slot** holding the xmin horizon — VACUUM runs but can free nothing.
Symptom: `last_autovacuum` is recent yet `n_dead_tup` keeps climbing. **This is the most commonly missed cause.**
2. **Scale factor too high** for a large table — the threshold is never reached.
3. **Too few workers** — three workers cannot keep up with hundreds of busy tables.
4. **Cost limits throttling it** — `autovacuum_vacuum_cost_delay` too high, so it sleeps more than it works.
5. **Someone disabled it** on the table: `ALTER TABLE ... SET (autovacuum_enabled = false)`.

```
-- Make autovacuum work harder on a busy system
ALTER SYSTEM SET autovacuum_max_workers = 6;           -- needs restart
ALTER SYSTEM SET autovacuum_vacuum_cost_limit = 2000; -- default 200
ALTER SYSTEM SET autovacuum_vacuum_cost_delay = '2ms';
ALTER SYSTEM SET autovacuum_naptime = '15s';
ALTER SYSTEM SET log_autovacuum_min_duration = '1s'; -- log what it does
SELECT pg_reload_conf();
```

3.8 When would you manually run VACUUM ANALYZE?

Autovacuum handles steady-state operation. Run it by hand when you have changed a lot of data **faster than the thresholds can react**:

- **Immediately after a bulk load, restore or large ETL run.** Statistics are stale and the first queries will get terrible plans.
- **After a mass `DELETE` or `UPDATE`** — do not wait for the 20% threshold.
- **Before a maintenance window ends**, so the system starts the working day with current statistics.
- **After a major version upgrade with `pg_upgrade`** — it does **not** carry statistics across. This is mandatory, not optional.
- **Before performance testing**, so results are not distorted by stale plans.
- **When troubleshooting a bad plan** — run `ANALYZE` and re-check before changing anything else.

```
-- One table
VACUUM (ANALYZE, VERBOSE) xxxx_orders;

-- Whole database, parallelised - use after a restore
vacuumdb --analyze --jobs=8 --echo xxxx

-- Statistics only, no vacuum - much cheaper
ANALYZE xxxx_orders;

-- Raise the sampling detail on a column with skewed data
ALTER TABLE xxxx_orders ALTER COLUMN status SET STATISTICS 1000;
ANALYZE xxxx_orders;
```

3.9 Why can VACUUM FULL be dangerous on a production database?

Three reasons, any one of which can cause an outage:

RISK	WHAT HAPPENS
Total table lock	ACCESS EXCLUSIVE for the whole operation. Every query on that table blocks — including SELECT . On a large table this is measured in hours.
Double disk requirement	It writes a complete new copy before dropping the old one. A 400 GB table needs 400 GB free. Starting one on a 90%-full disk takes the database down.
Lock queue cascade	While it waits to acquire its lock, every subsequent query queues behind it — so the table freezes before the rebuild even starts.

```
-- If you truly must, protect yourself
SET lock_timeout = '10s';          -- fail fast rather than queue behind a long query
VACUUM FULL VERBOSE xxxx_orders;

-- Far better in production
pg_repack -d xxxx -t public.xxxx_orders --jobs 4 --no-superuser-check
```

THE INTERVIEW ANSWER

“I would not run VACUUM FULL on a live production table. I would use pg_repack , which achieves the same compaction online with only a brief lock at the swap. If pg_repack is unavailable, I would schedule VACUUM FULL in an agreed maintenance window, confirm free disk exceeds the table size, and set a lock_timeout .”

4 Backup and Recovery

The one area where being wrong is unrecoverable. Everything else can be tuned later.

4.1 What is the difference between a logical and a physical backup?

	LOGICAL (<code>PG_DUMP</code>)	PHYSICAL (<code>PG_BASEBACKUP</code>)
What it copies	SQL statements and data that recreate objects	The actual data files, byte for byte
Granularity	Database, schema, single table	The whole cluster only
Portable across versions	Yes	No — same major version
Portable across architectures	Yes	No — same platform and word size
Speed to take	Slow on large databases	Fast — sequential file copy
Speed to restore	Very slow — replays SQL, rebuilds indexes	Fast — copy files back and start
Supports PITR	No	Yes , with WAL archiving
Output size	Smaller (no indexes, no bloat)	Full size including indexes and bloat
Best for	Migrations, single-table recovery, version upgrades	Disaster recovery, standby build, PITR

THE CORRECT ANSWER TO “WHICH SHOULD I USE?”

Both. Physical backups with WAL archiving give you fast full-cluster recovery and point-in-time capability. Periodic logical dumps give you the ability to restore one table that someone truncated, and give you a version-portable escape hatch. They solve different problems.

4.2 What is the difference between `pg_dump` and `pg_basebackup`?

	<code>PG_DUMP</code>	<code>PG_BASEBACKUP</code>
Type	Logical	Physical
Scope	One database (or part of one)	Entire cluster
Roles and tablespaces	Not included — need <code>pg_dumpall -g</code> <code>globals-only</code>	Included
Consistency	Single MVCC snapshot	Consistent via backup label and WAL
Connection	Normal client connection	Replication connection
Used to build a standby	No	Yes

```
-- LOGICAL: always use custom format, never plain SQL for anything sizeable
pg_dump -h HOST -U postgres -d xxxx -Fc -Z6 -j 4 -f xxxx_full.dump
pg_dumpall -h HOST -U postgres --globals-only -f globals.sql -- roles, do not skip

-- Inspect without restoring
pg_restore -l xxxx_full.dump | head -40

-- Restore, parallel
pg_restore -h HOST -U postgres -d xxxx_new -j 8 --verbose xxxx_full.dump

-- Restore ONE table out of a full dump
pg_restore -h HOST -U postgres -d xxxx -t xxxx_orders --data-only xxxx_full.dump
```

```
-- PHYSICAL
pg_basebackup -h HOST -U replicator -D /backup/base/full_20260909 \
    -Ft -z -Xs -P -v -c fast

-- Plain format, required if you want PG17 incremental backups later
pg_basebackup -h HOST -U replicator -D /backup/base/full_20260909 -c fast -P -v

-- Verify it (always)
pg_verifybackup /backup/base/full_20260909 -- expect: backup successfully verified
```

4.3 How do you take a backup of a 2 TB production database?

Answer. Not with `pg_dump`. At that size the restore time, not the backup time, is what disqualifies it — a 2 TB logical restore rebuilding every index can take a day or more, which no RTO will accept.

RANK	APPROACH	WHY
1	pgBackRest	Parallel, compressed, incremental and differential, block-level delta, built-in verification and retention, encrypted, S3-capable. The professional answer.
2	Storage snapshot (EBS, SAN, ZFS, LVM)	Near-instant. Wrap with <code>pg_backup_start()</code> / <code>pg_backup_stop()</code> , or ensure the snapshot is crash-consistent across all volumes.
3	pg_basebackup + WAL archive	Works, no extra software, but single-threaded and full every time.
4	<code>pg_dump</code>	Not for 2 TB. Keep it for per-table logical recovery only.

```
# pgBackRest - the practical answer for 2 TB
pgbackrest --stanza=xxxx stanza-create
pgbackrest --stanza=xxxx check
pgbackrest --stanza=xxxx --type=full --process-max=8 backup # weekly
pgbackrest --stanza=xxxx --type=diff --process-max=8 backup # daily
pgbackrest --stanza=xxxx --type=incr --process-max=8 backup # hourly
pgbackrest --stanza=xxxx info
```

```
# Snapshot method, low-level API (PG 15+ names)
SELECT pg_backup_start('nightly', fast => true);
-- ... take the storage snapshot of ALL volumes here ...
SELECT * FROM pg_backup_stop(); -- returns the backup_label contents - SAVE IT
```

```
# PostgreSQL 17: native incremental
pg_basebackup -D /backup/full -c fast -P # requires summarize_wal = on
pg_basebackup -D /backup/incr1 --incremental=/backup/full/backup_manifest -c fast -P
pg_combinebackup /backup/full /backup/incr1 -o /var/lib/pgsql/17/data # to restore
```

STATE YOUR RTO BEFORE CHOOSING A METHOD

"How would you back up 2 TB?" is really "how fast must you restore it?". A four-hour RTO rules out logical restore immediately and points at snapshots or pgBackRest with delta restore. Say that in the interview.

4.4 How do you restore a PostgreSQL backup?**Logical restore**

```
createdb -h HOST -U postgres xxxx_restore
psql -h HOST -U postgres -d xxxx_restore -f globals.sql          # roles first
pg_restore -h HOST -U postgres -d xxxx_restore -j 8 --verbose xxxx_full.dump
psql -h HOST -U postgres -d xxxx_restore -c "ANALYZE;"          # mandatory afterwards
```

Physical restore

```
systemctl stop postgresql
mv /var/lib/pgsql/17/data /var/lib/pgsql/17/data.old
mkdir -p /var/lib/pgsql/17/data && chmod 700 /var/lib/pgsql/17/data
tar -xzf /backup/base/full_20260909/base.tar.gz -C /var/lib/pgsql/17/data
tar -xzf /backup/base/full_20260909/pg_wal.tar.gz -C /var/lib/pgsql/17/data/pg_wal
chown -R postgres:postgres /var/lib/pgsql/17/data
systemctl start postgresql
```

pgBackRest restore

```
systemctl stop postgresql
pgbackrest --stanza=xxxx --delta restore          # delta = only changed files, much faster
systemctl start postgresql
```

THREE THINGS PEOPLE FORGET

Restore the **globals** before the data, or every object comes back owned by the wrong role. Run **ANALYZE** after a logical restore, or the first day is inexplicably slow. And check [postgresql.conf](#) and [pg_hba.conf](#) in the restored directory — a physical restore brings the *source* server's configuration with it.

4.5 What is Point-in-Time Recovery (PITR)?

Answer. Restore a base backup, then replay archived WAL **up to a chosen moment** — typically the second before a mistake. It is the only way to recover from "someone ran a DELETE without a WHERE clause at 14:03".

```
Base backup          WAL segments archived continuously
|                   |----|----|----|----|----|----|
Sun 02:00           Mon ..... 14:03 ..... now
|                   ^
+-----+ replay -----+
                        recovery_target_time = '14:02:59'
```

Prerequisites, configured in advance

```
-- postgresql.conf on the PRIMARY, before you ever need it
wal_level = replica
archive_mode = on
archive_command = 'test ! -f /wal_archive/%f && cp %p /wal_archive/%f'
archive_timeout = '300s'          -- force a segment switch at least every 5 min
```

Performing the recovery

```
# 1. Stop the server, move the damaged data directory aside
systemctl stop postgresql
mv /var/lib/pgsql/17/data /var/lib/pgsql/17/data.broken

# 2. Restore the base backup taken BEFORE the target time
tar -xzf /backup/base/full_20260906/base.tar.gz -C /var/lib/pgsql/17/data

# 3. Set the recovery target in postgresql.conf
# (recovery.conf was removed in PostgreSQL 12)
restore_command      = 'cp /wal_archive/%f %p'
recovery_target_time  = '2026-09-09 14:02:59+05:30'
recovery_target_action = 'pause'      -- pause so you can verify BEFORE promoting

# 4. Signal recovery mode and start
touch /var/lib/pgsql/17/data/recovery.signal
chown -R postgres:postgres /var/lib/pgsql/17/data
systemctl start postgresql
```

```
-- 5. Verify the data is as you expect, THEN promote
SELECT pg_is_in_recovery();           -- true while paused
SELECT count(*) FROM xxxx_orders;     -- confirm the rows are back
SELECT pg_wal_replay_resume();        -- or:
SELECT pg_promote();                  -- ends recovery, opens read-write
```

Other recovery targets: `recovery_target_xid`, `recovery_target_lsn`, `recovery_target_name` (set earlier with `pg_create_restore_point('before_release')`), and `recovery_target = 'immediate'` to stop as soon as consistency is reached.

4.6 What are WAL files, and what is WAL archiving?

WAL = Write-Ahead Log. Before any change reaches a data file, it is written to the WAL. This is what makes PostgreSQL crash-safe: after an unexpected shutdown, the server replays WAL to bring the data files back to a consistent state.

- Stored in `$PGDATA/pg_wal/`, default segment size **16 MB**.
- Named as 24 hex characters: timeline (8) + logical id (8) + segment (8), e.g. `000000010000000300000017`.
- Recycled and reused once no longer needed — the directory does not grow indefinitely under normal operation.

WAL archiving copies each completed segment somewhere durable before it is recycled. Those archived segments are what PITR and standby catch-up replay.

```
-- Simple local archive
archive_mode      = on
archive_command   = 'test ! -f /wal_archive/%f && cp %p /wal_archive/%f'

-- To S3 via pgBackRest (production)
archive_command   = 'pgbackrest --stanza=xxxx archive-push %p'
```

```
-- Is archiving actually working?
SELECT archived_count, last_archived_wal, last_archived_time,
       failed_count, last_failed_wal, last_failed_time, stats_reset
FROM   pg_stat_archiver;
```

THE RULE THAT SAVES YOU

`archive_command` must return **zero on success and non-zero on failure**, and must **never overwrite an existing file**. The `test ! -f` prefix is not decoration — without it a retry can silently corrupt the archive. And if the command keeps failing, PostgreSQL keeps every WAL segment until it succeeds, which fills `pg_wal` and eventually stops the database. Monitor `failed_count`.

4.7 How would you recover a database accidentally dropped at 2:00 PM?

Answer. PITR to 13:59:59. This is the textbook scenario and the interviewer wants the sequence, calmly.

```
# STEP 0 - STOP. Do not restart the server, do not "try things".
#           Preserve the current pg_wal directory; it may hold unarchived segments.
cp -a /var/lib/postgresql/17/data/pg_wal /safe/pg_wal_rescue

# STEP 1 - Identify the most recent base backup taken BEFORE 14:00
pgbackrest --stanza=xxxx info
ls -la /backup/base/

# STEP 2 - Confirm the WAL archive is continuous up to 14:00
ls /wal_archive/ | tail -20
psql -c "SELECT last_archived_wal, last_archived_time FROM pg_stat_archiver;"

# STEP 3 - Restore to a SEPARATE location. Never overwrite the live cluster.
systemctl stop postgresql
mkdir -p /var/lib/postgresql/17/recovery && chmod 700 /var/lib/postgresql/17/recovery
tar -xzf /backup/base/full_20260909/base.tar.gz -C /var/lib/postgresql/17/recovery

# STEP 4 - Recovery target: just before the drop
cat >> /var/lib/postgresql/17/recovery/postgresql.conf <<'EOF'
restore_command      = 'cp /wal_archive/%f %p'
recovery_target_time  = '2026-09-09 13:59:59+05:30'
recovery_target_action = 'pause'
port = 5433
EOF
touch /var/lib/postgresql/17/recovery/recovery.signal
chown -R postgres:postgres /var/lib/postgresql/17/recovery

# STEP 5 - Start on the alternate port and VERIFY before promoting
pg_ctl -D /var/lib/postgresql/17/recovery start
psql -p 5433 -c "\l"                # is the database there?
psql -p 5433 -d xxxx -c "SELECT count(*) FROM xxxx_orders;"

# STEP 6 - Extract just what you lost, or promote the whole cluster
pg_dump -p 5433 -d xxxx -Fc -f /tmp/recovered_xxxx.dump
# ... then restore into the live cluster ...
# OR, if the whole database is to be replaced:
psql -p 5433 -c "SELECT pg_promote();"

```

WHY THE ALTERNATE PORT MATTERS

Recovering alongside the live cluster rather than over it means that if the recovery target is wrong you simply adjust it and try again. Restoring directly over production gives you one attempt and no way back.

4.8 What are RPO and RTO?

TERM	DEFINITION	THE QUESTION IT ANSWERS
RPO Recovery Point Objective	The maximum acceptable amount of data loss , expressed as time	<i>"How much work may we lose?"</i>

TERM	DEFINITION	THE QUESTION IT ANSWERS
RTO Recovery Time Objective	The maximum acceptable downtime before service is restored	<i>"How long may we be down?"</i>

STRATEGY	TYPICAL RPO	TYPICAL RTO
Nightly <code>pg_dump</code> only	Up to 24 hours	Hours to a day
Base backup + WAL archive (PITR)	Minutes	1–4 hours
PITR + asynchronous standby	Seconds	Minutes
Synchronous standby + automatic failover	Zero	Seconds

```
-- Measure your ACTUAL RPO on a replicated system, do not assume it
SELECT application_name,
       pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), replay_lsn)) AS behind,
       replay_lag
FROM   pg_stat_replication;

-- Measure your actual archive RPO
SELECT now() - last_archived_time AS archive_rpo FROM pg_stat_archiver;
```

THE POINT OF THE QUESTION

RPO and RTO are **business decisions, not technical ones**. The DBA's job is to state what each architecture delivers and what it costs, then implement whichever the business signs off. An interviewer asking this wants to hear that you know the difference and that you measure the achieved figures rather than assuming the designed ones.

4.9 How do you verify that your backup is actually recoverable?

Answer. By restoring it. Every other check is a proxy. "The backup job reported success" is not verification.

Layered verification, cheapest first

```
# 1. Structural verification of a physical backup
pg_verifybackup /backup/base/full_20260909
#    expect exactly: backup successfully verified

# 2. Table of contents of a logical dump
pg_restore -l xxxx_full.dump | wc -l
pg_restore -l xxxx_full.dump | head -30

# 3. pgBackRest built-in verification
pgbackrest --stanza=xxxx verify
pgbackrest --stanza=xxxx info

# 4. Archive continuity - no gaps in the WAL sequence
ls /wal_archive/ | sort | awk 'NR>1 && $0 != prev {print "check:", prev, "->", $0} {prev=$0}'

# 5. THE REAL TEST - automated monthly restore rehearsal
pgbackrest --stanza=xxxx --delta --pg1-path=/restore/test restore
pg_ctl -D /restore/test -o "-p 5433" start
psql -p 5433 -d xxxx -c "SELECT count(*) FROM xxxx_orders;"
psql -p 5433 -d xxxx -c "SELECT max(created_on) FROM xxxx_orders;"
pg_ctl -D /restore/test stop && rm -rf /restore/test
```

SAY THIS IN THE INTERVIEW

“A backup that has never been restored is not a backup, it is a hope. I schedule a monthly automated restore into a scratch environment, run row-count and latest-timestamp assertions against it, record the elapsed restore time as the measured RTO, and alert if the rehearsal fails.” That answer distinguishes a DBA from a script operator.

4.10 What happens if the WAL archive gets full?

Answer. This is a two-stage failure and understanding the sequence is the point of the question.

```
Archive destination fills
|
archive_command starts FAILING (non-zero exit)
|
PostgreSQL REFUSES to recycle those WAL segments <-- by design, to protect PITR
|
pg_wal/ on the database server grows without bound
|
$PGDATA filesystem reaches 100%
|
PANIC: could not write to file - No space left on device
|
DATABASE SHUTS DOWN
```

```
-- Detect it before it becomes an outage
SELECT archived_count, failed_count, last_failed_wal, last_failed_time
FROM   pg_stat_archiver;

-- How much WAL is queued waiting to be archived?
SELECT count(*) AS pending_segments
FROM   pg_ls_dir('pg_wal/archive_status') f
WHERE  f LIKE '%.ready';

-- Size of pg_wal right now
SELECT pg_size_pretty(sum((pg_stat_file('pg_wal/'||name)).size))
FROM   pg_ls_dir('pg_wal') name WHERE name ~ '^[0-9A-F]{24}$';
```

Resolution, in order

1. **Free space at the archive destination** — move older segments to secondary storage, extend the volume, fix the S3 credentials. Archiving resumes automatically and `pg_wal` drains by itself.
2. If `pg_wal` is about to fill and you cannot free the archive fast enough, buy time by adding disk, or temporarily point `archive_command` at a different location with space.
3. Once healthy, clean the archive properly with retention, never by hand:

```
pg_archivecleanup /wal_archive 0000000100000000300000017
pgbackrest --stanza=xxxx expire
```

NEVER DO THIS

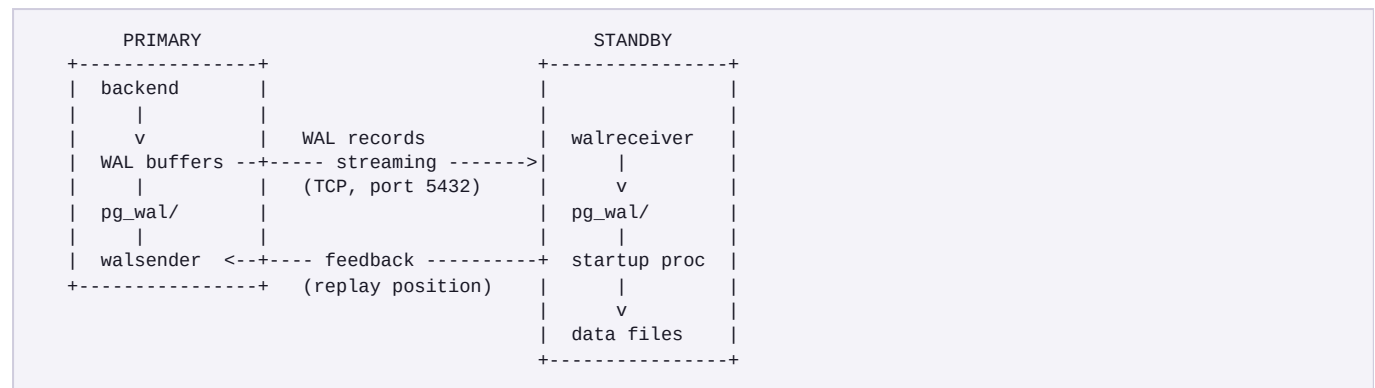
Do **not** delete files from `$PGDATA/pg_wal/` manually. Those segments have not yet been archived or checkpointed; deleting them corrupts the cluster and breaks every backup that depended on them. Setting `archive_command = '/bin/true'` to “make the error go away” is equally destructive — it silently discards WAL and invalidates your entire PITR chain from that moment.

5 Streaming Replication

How a standby stays current, how it falls behind, and what to do when it does.

5.1 What is streaming replication?

Answer. The primary streams WAL records to a standby over a replication connection as they are generated. A WAL sender process on the primary talks to a WAL receiver on the standby, which writes the WAL to disk and a startup process replays it into the data files. The standby is a byte-identical physical copy, continuously kept current, and can serve read-only queries (a **hot standby**).



Contrast with **logical replication**, which sends decoded row changes rather than physical WAL, can replicate selected tables, and works across major versions — but does not replicate DDL.

5.2 How do you configure primary to standby replication?

On the primary

```
-- postgresql.conf
wal_level = replica
max_wal_senders = 10
wal_keep_size = '2GB'           -- fallback if no slot is used
hot_standby = on
archive_mode = on
archive_command = 'test ! -f /wal_archive/%f && cp %p /wal_archive/%f'
```

```
-- Replication role
CREATE ROLE replicator WITH REPLICATION LOGIN PASSWORD 'strong-password';

-- A replication slot so the primary never discards WAL the standby still needs
SELECT pg_create_physical_replication_slot('standby1_slot');
```

```
# pg_hba.conf on the primary
host      replication    replicator      10.20.30.41/32    scram-sha-256
```

```
SELECT pg_reload_conf();  -- wal_level and max_wal_senders need a RESTART
```

On the standby

```
# 1. Take the base backup (this writes the connection settings for you)
rm -rf /var/lib/pgsql/17/data/*
pg_basebackup -h primary.host -U replicator -D /var/lib/pgsql/17/data \
              -Xs -P -v -R -C -S standby1_slot

# -R writes primary_conninfo into postgresql.auto.conf and creates standby.signal
# -C creates the replication slot named by -S if it does not exist
```

```
# 2. Confirm what -R wrote
cat /var/lib/pgsql/17/data/postgresql.auto.conf
# primary_conninfo = 'host=primary.host port=5432 user=replicator ...'
# primary_slot_name = 'standby1_slot'
ls /var/lib/pgsql/17/data/standby.signal      # must exist

# 3. Start
systemctl start postgresql
```

Verify from both sides

```
-- On the primary
SELECT application_name, client_addr, state, sync_state,
       sent_lsn, write_lsn, flush_lsn, replay_lsn
FROM   pg_stat_replication;

-- On the standby
SELECT pg_is_in_recovery();           -- must be true
SELECT pg_last_wal_receive_lsn(), pg_last_wal_replay_lsn();
```

5.3 What is the difference between synchronous and asynchronous replication?

	ASYNCHRONOUS (DEFAULT)	SYNCHRONOUS
Commit behaviour	Primary commits and returns immediately	Primary waits for standby acknowledgement
Data loss on failover	Possible — whatever had not reached the standby	None at the configured level
Commit latency	Unaffected	+ one network round trip, every commit
If the standby dies	Primary carries on	Primary blocks on commit unless another sync standby exists
Suits	Cross-region DR, read scaling	Same-site zero-loss requirement

```
-- Turn on synchronous replication (primary)
ALTER SYSTEM SET synchronous_standby_names = 'FIRST 1 (standby1, standby2)';
ALTER SYSTEM SET synchronous_commit = 'on';
SELECT pg_reload_conf();
```

```
-- Quorum form: any 2 of these 3 must acknowledge
ALTER SYSTEM SET synchronous_standby_names = 'ANY 2 (s1, s2, s3)';
```

SYNCHRONOUS_COMMIT	PRIMARY WAITS UNTIL THE STANDBY HAS...
off	nothing — not even the local WAL flush is waited for
local	local WAL flush only, standby ignored
remote_write	written to the standby OS (survives a PostgreSQL crash, not an OS crash)

SYNCHRONOUS_COMMIT	PRIMARY WAITS UNTIL THE STANDBY HAS...
on (default)	flushed to standby disk
remote_apply	replayed and visible to readers on the standby

THE TRAP WITH A SINGLE SYNCHRONOUS STANDBY

If your only synchronous standby goes down, **every commit on the primary hangs**. Zero data loss has been chosen over availability. Either run at least two sync candidates with `ANY 1 (s1, s2)`, or have a documented, rehearsed procedure to clear `synchronous_standby_names` under an outage.

5.4 How do you check replication status?

On the primary

```
SELECT pid, username, application_name, client_addr, state, sync_state,
       sent_lsn, write_lsn, flush_lsn, replay_lsn,
       write_lag, flush_lag, replay_lag,
       pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), replay_lsn)) AS bytes_behind
FROM   pg_stat_replication;
```

COLUMN	MEANING
state	streaming is healthy. catchup means it is still catching up. startup / backup are transient.
sync_state	async, sync, quorum or potential.
sent_lsn	How far the primary has sent.
write_lsn	How far the standby has written to its OS.
flush_lsn	How far the standby has fsynced. This determines data-loss exposure.
replay_lsn	How far the standby has applied. This determines read staleness and failover time.

On the standby

```
SELECT pg_is_in_recovery()           AS in_recovery,
       pg_last_wal_receive_lsn()      AS received,
       pg_last_wal_replay_lsn()       AS replayed,
       pg_last_xact_replay_timestamp() AS last_replayed_at,
       now() - pg_last_xact_replay_timestamp() AS time_lag;

-- Is the WAL receiver connected?
SELECT status, sender_host, sender_port, slot_name,
       latest_end_lsn, latest_end_time
FROM   pg_stat_wal_receiver;
```

5.5 How do you calculate replication lag?

There are two lags and they answer different questions. Measure both.

```
-- BYTE lag, measured on the PRIMARY - how much data is at risk
SELECT application_name, client_addr,
       pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), sent_lsn)) AS send_lag,
       pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), flush_lsn)) AS flush_lag,
       pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), replay_lsn)) AS replay_lag
FROM   pg_stat_replication;
```

```
-- TIME lag, measured on the STANDBY - how stale a read is
SELECT CASE
    WHEN pg_last_wal_receive_lsn() = pg_last_wal_replay_lsn() THEN 0
    ELSE EXTRACT(epoch FROM now() - pg_last_xact_replay_timestamp())
END AS lag_seconds;
```

WHY THE CASE MATTERS

On an idle primary no new WAL is produced, so `pg_last_xact_replay_timestamp()` stays where it was and naive time-lag arithmetic climbs steadily — producing a false alarm on a perfectly healthy standby. The `CASE` returns zero when received equals replayed, which means fully caught up. Every good monitoring check uses this form.

```
-- Interpreting the two together
-- flush_lag high, replay_lag high -> network or primary is outrunning the link
-- flush_lag ~0, replay_lag high -> standby received it but cannot APPLY fast enough
--                                     (usually a conflicting query, or slow disk)
```

5.6 The standby is showing replication lag. How will you troubleshoot it?

Start by deciding **which** lag it is, because the causes are entirely different.

```
-- Step 1: received vs replayed on the standby
SELECT pg_last_wal_receive_lsn() AS received,
       pg_last_wal_replay_lsn() AS replayed,
       pg_wal_lsn_diff(pg_last_wal_receive_lsn(), pg_last_wal_replay_lsn()) AS bytes_to_apply;
```

FINDING	MEANING	CAUSE TO INVESTIGATE
received < primary's current LSN	Network / transport lag	Bandwidth, latency, packet loss, saturated link, primary generating WAL faster than the link carries
received >> replayed	Apply lag	Recovery conflict with a long query on the standby, slow standby storage, single-threaded replay bottleneck

```
-- Apply lag: is a query on the standby blocking recovery?
SELECT pid, wait_event_type, wait_event, now()-query_start AS runtime, left(query,80)
FROM   pg_stat_activity WHERE state = 'active';

-- Recovery conflicts recorded so far
SELECT datname, confl_tablespace, confl_lock, confl_snapshot,
       confl_bufferpin, confl_deadlock
FROM   pg_stat_database_conflicts;

-- The startup process itself
SELECT pid, wait_event_type, wait_event, state
FROM   pg_stat_activity WHERE backend_type = 'startup';
```

Remedies by cause

- **Long queries on the standby blocking replay** — the usual culprit. Either accept staleness with `max_standby_streaming_delay`, or enable `hot_standby_feedback = on` (which instead causes bloat on the *primary*) — you must choose which side pays.
- **Network** — enable `wal_compression = on` on the primary; check MTU and link saturation.
- **Standby I/O** — replay is largely single-threaded; slow disks on the standby cannot be fixed by configuration.
- **Primary burst** — a bulk load or index rebuild generated WAL faster than the link. Often self-correcting; verify it recovers.
- **Checkpoint storms on the standby** — raise `max_wal_size` there too.

5.7 What happens if the standby server goes down for several hours?

Answer. It depends entirely on whether you configured a replication slot, and this is exactly what the question is testing.

CONFIGURATION	WHAT HAPPENS WHILE THE STANDBY IS DOWN
Replication slot in use	The primary retains every WAL segment the standby has not confirmed. The standby reconnects and catches up automatically. Risk: <code>pg_wal</code> grows without bound and can fill the primary's disk.
No slot, <code>wal_keep_size</code> set	The primary keeps that much WAL. If the standby is down longer than that covers, the needed segments are recycled and the standby cannot catch up: <code>requested WAL segment has already been removed</code> .
No slot, WAL archiving on	The standby fetches the missing segments from the archive via <code>restore_command</code> and catches up. This is the resilient design.
Neither	The standby is unrecoverable and must be rebuilt from a fresh base backup.

```
-- Guard against unbounded retention (PostgreSQL 13+)
ALTER SYSTEM SET max_slot_wal_keep_size = '50GB';
SELECT pg_reload_conf();

-- Now a slot that exceeds the cap is marked 'lost' rather than filling the disk
SELECT slot_name, active, wal_status,
       pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)) AS retained
FROM   pg_replication_slots;
-- wal_status: reserved | extended | unreserved | lost
```

THE DESIGN THAT SURVIVES BOTH FAILURE MODES

Use a replication slot **and** `max_slot_wal_keep_size` **and** WAL archiving with a `restore_command` on the standby. The slot handles short outages seamlessly; the cap protects the primary's disk from a long one; the archive lets the standby recover from a long outage without a rebuild.

5.8 What is a replication slot, and what if it retains too much WAL?

A **replication slot** is a named, server-side record of how far a particular consumer has consumed WAL. Its guarantee: the primary will **not remove WAL that the slot has not yet confirmed**, and it will not remove rows still needed by that consumer.

TYPE	USED BY
Physical slot	Streaming replication standbys
Logical slot	Logical replication, CDC tools such as Debezium

```
SELECT pg_create_physical_replication_slot('standby1_slot');
SELECT pg_drop_replication_slot('standby1_slot');

-- Health of every slot
SELECT slot_name, slot_type, database, active, active_pid,
       wal_status, safe_wal_size,
       pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)) AS retained_wal
FROM   pg_replication_slots
ORDER BY pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn) DESC;
```

When a slot retains too much WAL

An inactive slot — a decommissioned standby, a CDC consumer that was switched off, a test slot someone forgot — causes the primary to retain WAL **forever**. `pg_wal` grows until the filesystem is full and the database PANICs and shuts down. A logical slot additionally holds back the xmin horizon, so **VACUUM stops reclaiming and the whole cluster bloats**.

```
-- Find the offender
SELECT slot_name, active, wal_status,
       pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)) AS retained,
       age(xmin) AS xmin_age
FROM   pg_replication_slots
WHERE  NOT active OR wal_status <> 'reserved';

-- If the consumer is genuinely gone, drop it. Space is freed at the next checkpoint.
SELECT pg_drop_replication_slot('old_unused_slot');
CHECKPOINT;
```

NUMBER ONE CAUSE OF “THE DISK FILLED UP OVERNIGHT”

An inactive replication slot. Whenever `pg_wal` is growing and you cannot explain it, `pg_replication_slots` is the first place to look — before checkpoints, before archiving, before anything else. Monitor `active = false` as an alerting condition.

5.9 How do you monitor replication health?

Six checks. Alert on all of them.

```
-- 1. Is a standby connected at all? (alert if 0)
SELECT count(*) AS connected_standbys FROM pg_stat_replication;

-- 2. Byte lag per standby (warn >100MB, critical >1GB)
SELECT application_name,
       pg_wal_lsn_diff(pg_current_wal_lsn(), replay_lsn) AS replay_lag_bytes
FROM   pg_stat_replication;

-- 3. Time lag on the standby (warn >60s, critical >300s)
SELECT CASE WHEN pg_last_wal_receive_lsn() = pg_last_wal_replay_lsn() THEN 0
       ELSE EXTRACT(epoch FROM now() - pg_last_xact_replay_timestamp()) END AS lag_s;

-- 4. Slot health (alert on inactive or non-reserved)
SELECT slot_name, active, wal_status,
       pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn) AS retained_bytes
FROM   pg_replication_slots;

-- 5. Archiving (alert if failed_count rises)
SELECT archived_count, failed_count, last_failed_wal, last_failed_time
FROM   pg_stat_archiver;

-- 6. Standby is actually in recovery (alert if false unexpectedly - split brain)
SELECT pg_is_in_recovery();
```

THE CHECK PEOPLE LEAVE OUT

Number 6. If a standby reports `pg_is_in_recovery() = false` and you did not promote it, you have **two primaries accepting writes**. That is split brain, and every minute it continues makes the data harder to reconcile. It should page someone immediately.

6 High Availability and Failover

What you do at 3 a.m., and what you should have built before then.

6.1 The primary PostgreSQL server goes down. What will you do?

Answer. Do not promote first. Diagnose first — a promotion you did not need is far harder to undo than thirty seconds of checking.

```
# STEP 1 - Is the server truly down, or just unreachable from here?
ping primary.host
psql -h primary.host -U postgres -c "SELECT 1"
ssh primary.host 'systemctl status postgresql'
ssh primary.host 'tail -100 /var/lib/pgsql/17/data/log/postgresql-*.log'
ssh primary.host 'df -h; free -m'
```

```
# STEP 2 - If PostgreSQL simply stopped, try to start it. This is often the whole fix.
systemctl start postgresql
# Check the log for the reason it stopped: disk full, OOM killer, PANIC, corruption.
```

```
# STEP 3 - If it cannot be recovered quickly, assess the standby BEFORE promoting
psql -h standby.host -c "SELECT pg_is_in_recovery();"
psql -h standby.host -c "SELECT pg_last_wal_receive_lsn(), pg_last_wal_replay_lsn(),
                        pg_last_xact_replay_timestamp();"
# How much data will be lost? That is your actual RPO for this incident.
```

```
# STEP 4 - FENCE the old primary. Non-negotiable.
ssh primary.host 'systemctl stop postgresql && systemctl disable postgresql'
# or shut down the VM / detach its network / STONITH.
# If you cannot reach it to fence it, be certain it cannot come back on its own.
```

```
# STEP 5 - Promote the standby
psql -h standby.host -c "SELECT pg_promote(wait => true);"
# or: pg_ctl promote -D /var/lib/pgsql/17/data
psql -h standby.host -c "SELECT pg_is_in_recovery();" -- must now be false
```

```
# STEP 6 - Repoint applications, then rebuild
# Move the VIP / update DNS / update the connection string / reconfigure PgBouncer.
# Then rebuild the old primary as a standby of the new one (see the pg_rewind question).
```

THE ORDER THAT MATTERS

Fence before you promote. Promoting while the old primary can still accept writes is how split brain happens. If you take one thing from this section, take that.

6.2 How do you perform a manual failover?

Planned switchover (both servers healthy)

```
# 1. Stop application writes
# 2. On the OLD PRIMARY - clean shutdown so all WAL is flushed and sent
pg_ctl -D /var/lib/pgsql/17/data stop -m fast

# 3. On the STANDBY - confirm it received everything
psql -c "SELECT pg_last_wal_receive_lsn(), pg_last_wal_replay_lsn();"
# Both should equal the old primary's final LSN. ZERO data loss.

# 4. Promote
psql -c "SELECT pg_promote(wait => true);"
psql -c "SELECT pg_is_in_recovery();"          -- false

# 5. Rebuild the old primary as the new standby
pg_rewind --target-pgdata=/var/lib/pgsql/17/data \
          --source-server="host=newprimary user=replicator dbname=postgres" -P
touch /var/lib/pgsql/17/data/standby.signal
# set primary_conninfo in postgresql.auto.conf, then start

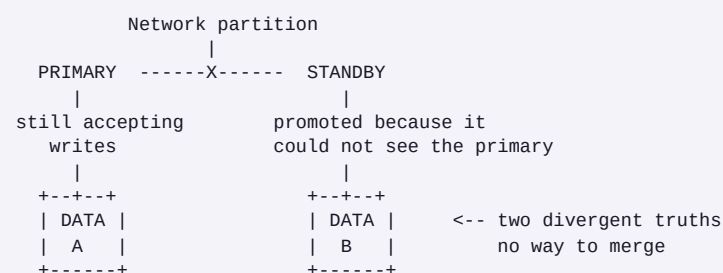
# 6. Repoint applications and verify replication in the new direction
```

Unplanned failover (primary is gone)

```
# 1. FENCE the old primary
# 2. Record the data-loss position for the incident report
psql -h standby -c "SELECT pg_last_wal_replay_lsn(), pg_last_xact_replay_timestamp();"
# 3. Promote
psql -h standby -c "SELECT pg_promote(wait => true);"
# 4. Repoint applications
# 5. Rebuild the old primary once recovered - pg_rewind if possible, base backup if not
```

6.3 What is split brain, and how do you prevent it?

Split brain is two servers both believing they are the primary and both accepting writes. The two datasets diverge immediately and there is no automatic way to merge them — recovery means choosing one and reconciling the other by hand, or discarding it.



Prevention, in order of strength

CONTROL	HOW IT WORKS
Fencing / STONITH	Forcibly power off or isolate the old primary before promotion. The only guaranteed method.
Quorum in a DCS	Patroni with etcd/Consul: a node may only be primary while it holds the leader key, which requires a majority. A minority partition cannot obtain it.
Watchdog	Hardware or software watchdog reboots a node that loses its leader lock, so it cannot linger as a second primary.

CONTROL	HOW IT WORKS
Odd number of nodes	Three or five DCS members so a majority always exists on exactly one side of any partition.
Single write endpoint	One VIP or HAProxy front end that can only point at one node at a time.
Synchronous replication	Limits how much can diverge, and makes reconciliation feasible. Not prevention on its own.

WHY MANUAL HA IS RISKY

Manual failover is safe only if the operator always fences before promoting. Automated failover without a quorum-backed DCS and fencing is **more** dangerous than manual, because it will happily promote during a transient network blip. If you automate, automate with quorum.

6.4 What tools can be used for PostgreSQL HA?

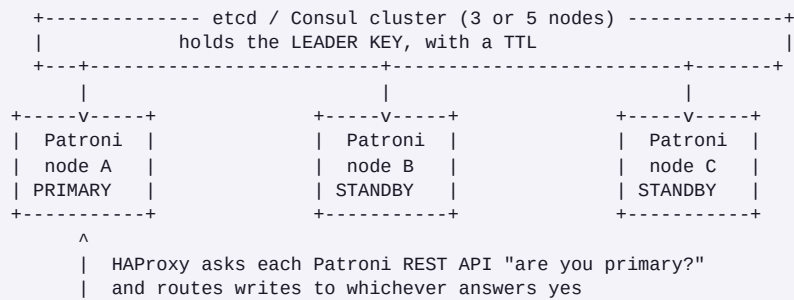
TOOL	MODEL	NOTES
Patroni	DCS-backed template (etcd, Consul, ZooKeeper, Kubernetes)	The de facto standard. Automatic failover, leader election by quorum, REST API, integrates with HAProxy. Kubernetes-native.
repmgr	Daemon plus command-line	Simpler than Patroni, no external DCS. Witness node for quorum. Good for small estates.
Pacemaker + Corosync	General-purpose cluster manager	With the PAF resource agent. Powerful, true STONITH, steep learning curve. Common in enterprise and RHEL environments.
pg_auto_failover	Monitor node plus keepers	From Microsoft. Simple three-node model, easy to reason about.
Stolon	Cloud-native, DCS-backed	Kubernetes oriented, proxy-based routing.
PgBouncer / HAProxy / pgpool-II	Connection routing	Not HA on their own. They route traffic to whichever node the HA layer says is primary.
Managed services	RDS, Aurora, Cloud SQL, Azure Flexible Server	The provider handles failover. You configure it, you do not operate it.

A COMPLETE STACK

Patroni for leader election and failover, **etcd** (three or five nodes) as the DCS, **HAProxy** for a single write endpoint plus a read endpoint, **pgBackRest** for backups and PITR, and **PgBouncer** for pooling. That combination is what most serious self-managed PostgreSQL estates converge on.

6.5 What is Patroni and how does it help with automatic failover?

Patroni is a Python daemon that runs alongside each PostgreSQL instance and manages its role. It does not replace PostgreSQL replication — it decides **who is primary** and enforces that decision through a distributed consensus store.



How failover works

1. The primary's Patroni renews the **leader key** in etcd every few seconds. The key has a TTL (default 30 s).
2. If the primary dies or is partitioned, it stops renewing and **the key expires**.
3. Standby Patronis notice, compare their WAL positions, and the most advanced eligible node **races to acquire the leader key**.
4. Only one can win, because etcd requires a quorum. The winner promotes PostgreSQL.
5. The others reconfigure to follow the new leader, using `pg_rewrite` if they had diverged.
6. HAProxy's health check sees the new primary and routes writes there.

Why this prevents split brain: a node in a minority partition **cannot obtain the leader key**, and Patroni demotes any PostgreSQL instance that finds itself primary without holding it.

```

# Everyday Patroni commands
patronictl -c /etc/patroni.yml list
patronictl -c /etc/patroni.yml switchover --master node-a --candidate node-b
patronictl -c /etc/patroni.yml failover --candidate node-b
patronictl -c /etc/patroni.yml reinit node-c           # rebuild a broken member
patronictl -c /etc/patroni.yml pause                  # suspend automation for maintenance
patronictl -c /etc/patroni.yml resume
patronictl -c /etc/patroni.yml edit-config            # change PostgreSQL params cluster-wide

```

6.6 After failover, how do you rebuild the old primary as a standby?

Answer. Use `pg_rewrite` when you can — it synchronises only the blocks that diverged, so it takes minutes rather than the hours a full base backup needs on a large database.

Prerequisites, which must be true beforehand

- `wal_log_hints = on` or the cluster was initialised with `--data-checksums`. Without one of these, `pg_rewrite` cannot work — and you cannot enable it after the fact without a restart.
- The old primary must be **cleanly shut down** first.
- A role with sufficient privileges on the new primary.

```
# 1. Ensure the old primary is stopped cleanly
pg_ctl -D /var/lib/pgsql/17/data stop -m fast
# If it crashed, start it and stop it cleanly so recovery completes first.

# 2. Rewind against the new primary
pg_rewind --target-pgdata=/var/lib/pgsql/17/data \
          --source-server="host=newprimary.host user=replicator dbname=postgres" \
          --progress --restore-target-wal

# 3. Configure it to follow the new primary
cat >> /var/lib/pgsql/17/data/postgresql.auto.conf <<'EOF'
primary_conninfo = 'host=newprimary.host port=5432 user=replicator password=xxx
application_name=old_primary'
primary_slot_name = 'old_primary_slot'
restore_command = 'cp /wal_archive/%f %p'
EOF
touch /var/lib/pgsql/17/data/standby.signal

# 4. Create the slot on the new primary, then start
psql -h newprimary.host -c "SELECT pg_create_physical_replication_slot('old_primary_slot');"
systemctl start postgresql

# 5. Verify
psql -c "SELECT pg_is_in_recovery();" # true, on the rebuilt node
psql -h newprimary.host -c "SELECT * FROM pg_stat_replication;"
```

```
# With Patroni this is one command
patronictl -c /etc/patroni.yml reinit old-primary-node
```

IF PG_REWIND IS NOT POSSIBLE

No `wal_log_hints`, no checksums, or the divergence is too great — then you rebuild from a fresh base backup:

```
rm -rf /var/lib/pgsql/17/data/*
pg_basebackup -h newprimary.host -U replicator -D /var/lib/pgsql/17/data -Xs -P -R -C -S
old_primary_slot
```

On a 2 TB database that is hours, not minutes. Which is why `wal_log_hints = on` belongs in your build standard from day one.

7 Disk Space Issues

Every DBA gets this page. The cause is nearly always one of five things.

7.1 PostgreSQL server disk usage suddenly reaches 95%. What will you check first?

Answer. Five suspects, in this order. The first two account for most incidents.

#	SUSPECT	TELL
1	WAL accumulation in <code>pg_wal/</code>	Inactive replication slot, or <code>archive_command</code> failing
2	Bloat from failed autovacuum	High <code>n_dead_tup</code> , old <code>last_autovacuum</code>
3	Log files	<code>log/</code> directory large; verbose logging left on after debugging
4	Temporary files	<code>base/pgsql_tmp/</code> large; a big sort spilling with <code>work_mem</code> too small
5	Genuine data growth	Everything else is normal and the tables really are bigger

```
# Where is the space going? Start at the OS.
df -h
du -sh /var/lib/pgsql/17/data/* | sort -h | tail -15
du -sh /var/lib/pgsql/17/data/pg_wal
du -sh /var/lib/pgsql/17/data/log
du -sh /var/lib/pgsql/17/data/base/pgsql_tmp
```

```
-- Then from inside the database
SELECT pg_size_pretty(pg_database_size(current_database())) AS db_size;

-- WAL volume
SELECT count(*) AS wal_segments,
       pg_size_pretty(count(*) * 16 * 1024 * 1024) AS wal_size
FROM   pg_ls_waldir();

-- Slots retaining WAL - check this FIRST, it is the usual answer
SELECT slot_name, active, wal_status,
       pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)) AS retained
FROM   pg_replication_slots ORDER BY 4 DESC;

-- Archiver failing?
SELECT archived_count, failed_count, last_failed_wal, last_failed_time FROM pg_stat_archiver;
```

7.2 How do you find the largest databases and tables?

```
-- Largest databases
SELECT datname,
       pg_size_pretty(pg_database_size(datname)) AS size,
       pg_database_size(datname) AS bytes
FROM   pg_database
WHERE  datistemplate = false
ORDER BY pg_database_size(datname) DESC;
```

```
-- Largest tables, with the table/index/toast split - the version that is actually useful
SELECT n.nspname AS schema,
       c.relname AS name,
       pg_size_pretty(pg_total_relation_size(c.oid)) AS total,
       pg_size_pretty(pg_table_size(c.oid)) AS table_only,
       pg_size_pretty(pg_indexes_size(c.oid)) AS indexes,
       pg_size_pretty(COALESCE(pg_total_relation_size(c.reltoastrelid),0)) AS toast,
       c.reltuples::bigint AS approx_rows
FROM   pg_class c
JOIN   pg_namespace n ON n.oid = c.relnamespace
WHERE  c.relkind IN ('r','p','m')
      AND n.nspname NOT IN ('pg_catalog','information_schema')
ORDER  BY pg_total_relation_size(c.oid) DESC
LIMIT 20;
```

```
-- Largest indexes
SELECT schemaname, relname AS table, indexrelname AS index,
       pg_size_pretty(pg_relation_size(indexrelid)) AS size, idx_scan
FROM   pg_stat_user_indexes
ORDER  BY pg_relation_size(indexrelid) DESC LIMIT 20;

-- Size per schema
SELECT n.nspname, pg_size_pretty(sum(pg_total_relation_size(c.oid))) AS size
FROM   pg_class c JOIN pg_namespace n ON n.oid = c.relnamespace
GROUP  BY 1 ORDER BY sum(pg_total_relation_size(c.oid)) DESC;
```

READ THE SPLIT, NOT JUST THE TOTAL

If `indexes` exceeds `table_only` by a wide margin you have index bloat or redundant indexes. If `toast` dominates you are storing large values — documents, JSON, images — and the fix is architectural, not a `VACUUM`.

7.3 Why is WAL consuming huge disk space?

Four causes. Diagnose in this order.

RANK	CAUSE	CHECK
1	Inactive replication slot	<code>pg_replication_slots</code> where <code>active = false</code>
2	<code>archive_command</code> failing	<code>pg_stat_archiver.failed_count</code> rising
3	<code>wal_keep_size</code> set very high	<code>SHOW wal_keep_size;</code>
4	Checkpoints too infrequent	<code>max_wal_size</code> very large, or a genuine write burst

```
-- 1
SELECT slot_name, active, wal_status,
       pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)) AS retained
FROM   pg_replication_slots ORDER BY 4 DESC;

-- 2
SELECT archived_count, failed_count, last_archived_wal, last_archived_time,
       last_failed_wal, last_failed_time FROM pg_stat_archiver;
SELECT count(*) AS waiting_to_archive
FROM   pg_ls_dir('pg_wal/archive_status') f WHERE f LIKE '%.ready';

-- 3 and 4
SHOW wal_keep_size; SHOW max_wal_size; SHOW checkpoint_timeout;
SELECT * FROM pg_stat_bgwriter;      -- checkpoints_timed vs checkpoints_req
```

```
-- Immediate relief once the cause is fixed
CHECKPOINT; -- allows recycling of eligible segments
SELECT pg_drop_replication_slot('dead_slot'); -- if the consumer is genuinely gone
```

NEVER DELETE PG_WAL FILES BY HAND

Those segments have not been archived or checkpointed. Removing them corrupts the cluster and invalidates every backup that depended on them. Fix the cause and PostgreSQL recycles them itself.

7.4 How do replication slots cause disk space issues?

A replication slot's entire purpose is to **guarantee WAL is retained** until its consumer confirms receipt. If the consumer disappears and the slot is left behind, that guarantee has no end date.

```
Standby decommissioned, slot left in place
|
Slot's restart_lsn stops advancing
|
Primary retains EVERY WAL segment from that point onward
|
pg_wal/ grows: 16 MB, 16 GB, 160 GB ...
|
Filesystem full -> PANIC -> database down

A LOGICAL slot does one thing more:
|
xmin horizon is also held back
|
VACUUM cannot reclaim ANY dead tuple cluster-wide -> bloat everywhere
```

```
-- Detect
SELECT slot_name, slot_type, database, active, active_pid, wal_status,
       pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)) AS retained_wal,
       age(xmin) AS xmin_age, age(catalog_xmin) AS catalog_xmin_age
FROM   pg_replication_slots
ORDER BY pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn) DESC;

-- Resolve: confirm the consumer is truly gone, then drop
SELECT pg_drop_replication_slot('old_standby_slot');
CHECKPOINT;

-- Prevent (PostgreSQL 13+): cap retention so a dead slot cannot fill the disk
ALTER SYSTEM SET max_slot_wal_keep_size = '50GB';
SELECT pg_reload_conf();
```

THE TRADE-OFF IN MAX_SLOT_WAL_KEEP_SIZE

Once the cap is exceeded the slot is marked **lost** and its consumer can no longer catch up from WAL — it must be rebuilt or fall back to the archive. That is a deliberate choice: **lose one standby rather than the primary**. Set it, and set the archive up so the standby can still recover.

7.5 What would you do if /pg_wal reaches 100%?

Answer. This is an emergency — the database will PANIC and shut down, and it will not restart until there is space. Work fast and in this order.

```
# 1. BUY SPACE IMMEDIATELY - do not delete anything in pg_wal
#   Remove anything else on that filesystem first:
rm -f /var/lib/pgsql/17/data/log/postgresql-2026-0[1-7]*.log # old logs only
#   Extend the LVM volume / EBS volume if you can:
lvextend -L +20G /dev/vg0/pgdata && resize2fs /dev/vg0/pgdata
```

```
# 2. FIND THE CAUSE while space is being freed
psql -c "SELECT slot_name, active, wal_status,
               pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn))
               FROM pg_replication_slots ORDER BY 4 DESC;"
psql -c "SELECT failed_count, last_failed_wal, last_failed_time FROM pg_stat_archiver;"
```

```
# 3. FIX THE CAUSE
# a) Dead slot:
psql -c "SELECT pg_drop_replication_slot('dead_slot');"
# b) Archive destination full - free it, or repoint archive_command somewhere with space:
psql -c "ALTER SYSTEM SET archive_command = 'test ! -f /bigvol/wal/%f && cp %p /bigvol/wal/%f';"
psql -c "SELECT pg_reload_conf();"

```

```
# 4. RECLAIM
psql -c "CHECKPOINT;"          # eligible segments are recycled after this
```

If the database has already shut down

```
# Free space on the volume by ANY means other than deleting pg_wal.
# If pg_wal is on its own filesystem and you truly cannot extend it, you may
# MOVE (never delete) already-archived segments elsewhere, and only those
# confirmed present in the archive:
# for f in $(ls pg_wal | head -50); do
#     [ -f /wal_archive/$f ] && mv pg_wal/$f /emergency_space/
# done
# Then start the server and fix the root cause properly.
```

PREVENTION IS ONE ALERT

Alert when `pg_wal` exceeds twice `max_wal_size`, and alert on any slot with `active = false`. Both conditions appear hours before the disk fills. Nobody should ever meet this situation unwarned.

7.6 How do you identify table and index bloat, and what do you do about it?

```
-- Signal (free, instant)
SELECT relname, n_live_tup, n_dead_tup,
       round(100.0*n_dead_tup/NULLIF(n_live_tup+n_dead_tup,0),1) AS dead_pct,
       last_autovacuum,
       pg_size_pretty(pg_total_relation_size(relid)) AS size
FROM   pg_stat_user_tables
WHERE  n_dead_tup > 10000
ORDER BY n_dead_tup DESC LIMIT 20;

-- Measurement (accurate, scans the table)
CREATE EXTENSION IF NOT EXISTS pgstattuple;
SELECT * FROM pgstattuple('xxx_orders');
SELECT * FROM pgstatindex('idx_xxx_orders_created');
```

REMEDY	DOWNTIME	USE WHEN
VACUUM	None	Always first. Often enough on its own.
REINDEX CONCURRENTLY	None (PG 12+)	Index bloat specifically. Very common and often the whole problem.
pg_repack	Brief lock at swap	Production table bloat. Needs 2× the table size free.
VACUUM FULL	Full table lock	Maintenance window only.
Drop a partition	Brief	Best of all, when the table is partitioned on the purge key.

```
REINDEX INDEX CONCURRENTLY idx_xxxx_orders_created;  
REINDEX TABLE CONCURRENTLY xxxx_orders;  
pg_repack -d xxxx -t public.xxxx_orders --jobs 4
```

TREAT THE CAUSE, NOT THE SYMPTOM

Bloat that returns after repacking means autovacuum is being prevented from working — almost always a long transaction or an abandoned replication slot holding the xmin horizon. Repacking on a schedule to manage recurring bloat is treating a symptom.

8 User and Security Management

Roles, grants, and the file that decides whether anyone gets in at all.

8.1 What is the difference between a LOGIN role and a NOLOGIN role?

Answer. PostgreSQL has only **roles**. “User” and “group” are conventions, not separate object types. The `LOGIN` attribute is the only thing that distinguishes them.

	<code>LOGIN</code> ROLE (A “USER”)	<code>NOLOGIN</code> ROLE (A “GROUP”)
Can connect	Yes	No
Created by	<code>CREATE USER</code> or <code>CREATE ROLE ... LOGIN</code>	<code>CREATE ROLE</code> (default)
Purpose	A person or an application	A container for privileges, granted to other roles

```
-- CREATE USER is simply an alias for CREATE ROLE ... LOGIN
CREATE ROLE xxxx_app LOGIN PASSWORD 'strong-password';
CREATE ROLE xxxx_readonly;                -- NOLOGIN by default: a group

-- The correct pattern: grant privileges to the GROUP, membership to the USER
GRANT CONNECT ON DATABASE xxxx TO xxxx_readonly;
GRANT USAGE ON SCHEMA public TO xxxx_readonly;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO xxxx_readonly;
GRANT xxxx_readonly TO xxxx_app;          -- membership

-- Switch a role between the two
ALTER ROLE xxxx_app NOLOGIN;              -- disable access without dropping
ALTER ROLE xxxx_app LOGIN;
```

WHY THIS PATTERN MATTERS OPERATIONALLY

Grant to groups, never to individuals. When someone leaves you revoke one membership rather than auditing hundreds of object grants. And `ALTER ROLE ... NOLOGIN` disables an account instantly without dropping it — dropping a role that owns objects fails, and forcing it means reassigning ownership under pressure.

8.2 How do you create a user with read-only access?

```
-- 1. The group that holds the privileges
CREATE ROLE xxxx_readonly NOLOGIN;

-- 2. Connect and schema usage
GRANT CONNECT ON DATABASE xxxx TO xxxx_readonly;
\c xxxx
GRANT USAGE ON SCHEMA public TO xxxx_readonly;

-- 3. Read on everything that exists today
GRANT SELECT ON ALL TABLES    IN SCHEMA public TO xxxx_readonly;
GRANT SELECT ON ALL SEQUENCES IN SCHEMA public TO xxxx_readonly;

-- 4. THE STEP EVERYONE FORGETS: everything created in FUTURE
ALTER DEFAULT PRIVILEGES IN SCHEMA public
    GRANT SELECT ON TABLES TO xxxx_readonly;
ALTER DEFAULT PRIVILEGES IN SCHEMA public
    GRANT SELECT ON SEQUENCES TO xxxx_readonly;

-- 5. The actual login, as a member of the group
CREATE ROLE xxxx_reporting LOGIN PASSWORD 'strong-password'
    CONNECTION LIMIT 5 VALID UNTIL '2027-01-01';
GRANT xxxx_readonly TO xxxx_reporting;

-- 6. Belt and braces
ALTER ROLE xxxx_reporting SET default_transaction_read_only = on;
ALTER ROLE xxxx_reporting SET statement_timeout = '5min';
```

ALTER DEFAULT PRIVILEGES APPLIES PER GRANTOR

It only affects objects created **by the role that ran it**. If your application deploys tables as `xxxx_owner`, you must run it as that role (or use `FOR ROLE xxxx_owner`), otherwise every new table is invisible to the read-only user and someone re-runs the grants every release.

```
ALTER DEFAULT PRIVILEGES FOR ROLE xxxx_owner IN SCHEMA public
    GRANT SELECT ON TABLES TO xxxx_readonly;
```

```
-- Verify what a role can actually do
SELECT has_table_privilege('xxxx_reporting', 'public.xxxx_orders', 'SELECT') AS can_select,
       has_table_privilege('xxxx_reporting', 'public.xxxx_orders', 'INSERT') AS can_insert;
\dp public.xxxx_orders
```

8.3 How do you provide access to a specific schema or table, and what is the difference between GRANT and REVOKE?

```
-- Schema-scoped access
GRANT USAGE ON SCHEMA reporting TO xxxx_analyst; -- USAGE = "may look inside"
GRANT SELECT ON ALL TABLES IN SCHEMA reporting TO xxxx_analyst;

-- Single table, specific privileges
GRANT SELECT, INSERT, UPDATE ON xxxx_orders TO xxxx_app;

-- Column level
GRANT SELECT (order_id, status, created_on) ON xxxx_orders TO xxxx_analyst;
GRANT UPDATE (status) ON xxxx_orders TO xxxx_app;

-- Row level
ALTER TABLE xxxx_orders ENABLE ROW LEVEL SECURITY;
CREATE POLICY branch_isolation ON xxxx_orders
  FOR SELECT TO xxxx_app
  USING (branch_id = current_setting('app.branch_id')::int);
```

	GRANT	REVOKE
Effect	Gives a privilege	Takes a privilege away
Default state	Most privileges are absent by default	Revoking something never granted is a no-op, not an error
Cascading	<code>WITH GRANT OPTION</code> lets the grantee re-grant	<code>CASCADE</code> also removes privileges that grantee passed on
PUBLIC	Everyone implicitly holds PUBLIC 's privileges	Revoke from PUBLIC explicitly — per-role revokes will not help

```
-- Hardening every new database should have
REVOKE ALL ON DATABASE xxxx FROM PUBLIC;
REVOKE ALL ON SCHEMA public FROM PUBLIC;
REVOKE CREATE ON SCHEMA public FROM PUBLIC; -- default before PostgreSQL 15

-- Inspect
\dp public.xxxx_orders -- object privileges
\du -- roles and attributes
SELECT * FROM information_schema.role_table_grants WHERE grantee = 'xxxx_app';
```

8.4 How does pg_hba.conf work?

pg_hba.conf — Host-Based Authentication — decides **who may connect, from where, to what, and how they must prove it**. It is evaluated **before** any privilege check.

#	TYPE	DATABASE	USER	ADDRESS	METHOD
	local	all	postgres		peer
	host	xxxx	xxxx_app	10.20.30.0/24	scram-sha-256
	hostssl	xxxx	xxxx_report	10.20.40.15/32	scram-sha-256
	host	replication	replicator	10.20.30.41/32	scram-sha-256
	host	all	all	0.0.0.0/0	reject

FIELD	VALUES
TYPE	<code>local</code> (Unix socket), <code>host</code> (TCP, SSL or not), <code>hostssl</code> (SSL required), <code>hostnossl</code>
DATABASE	A name, <code>all</code> , <code>replication</code> , or a comma-separated list
USER	A role name, <code>all</code> , or <code>+groupname</code> for members of a group

FIELD	VALUES
ADDRESS	CIDR, hostname, or <code>samenet</code>
METHOD	<code>scram-sha-256</code> , <code>md5</code> , <code>trust</code> , <code>reject</code> , <code>peer</code> , <code>cert</code> , <code>ldap</code> , <code>gss</code>

THE RULE THAT CATCHES EVERYONE

The first matching line wins, and evaluation stops there. If a broad `host all all 0.0.0.0/0 md5` line sits above your specific rule, the specific rule is never reached. Order from most specific to least specific, always.

```
-- Apply changes (no restart needed)
SELECT pg_reload_conf();

-- Read the active rules from SQL, in evaluation order (PG 10+)
SELECT line_number, type, database, user_name, address, auth_method, error
FROM   pg_hba_file_rules ORDER BY line_number;
-- a non-null 'error' column means that line is malformed and being ignored
```

8.5 What is the difference between trust, md5 and scram-sha-256 authentication?

METHOD	HOW IT WORKS	VERDICT
<code>trust</code>	No password at all. Anyone who reaches the port and names a role is admitted as that role.	Never in production. Emergency local recovery only.
<code>md5</code>	Challenge-response over an MD5 hash of password + username. The stored hash is salted only with the username, so it is replayable and crackable.	Deprecated. Legacy clients only.
<code>scram-sha-256</code>	RFC 5802 SCRAM. Random per-user salt, iterated hashing, mutual proof, and the password never crosses the wire in any recoverable form.	The correct choice. Default since PostgreSQL 14.
<code>peer</code> / <code>ident</code>	Trusts the operating-system username. <code>peer</code> for local sockets, <code>ident</code> over TCP.	Fine locally; <code>ident</code> over TCP is not trustworthy.
<code>cert</code>	Client TLS certificate. No password.	Excellent for service-to-service.

```
-- Migrate from md5 to scram-sha-256
SHOW password_encryption;           -- want: scram-sha-256
ALTER SYSTEM SET password_encryption = 'scram-sha-256';
SELECT pg_reload_conf();

-- Existing passwords keep their old md5 hash until reset. Find them:
SELECT rolname,
       CASE WHEN rolpassword LIKE 'SCRAM-SHA-256%' THEN 'scram'
            WHEN rolpassword LIKE 'md5%' THEN 'md5 - RESET THIS'
            ELSE 'none' END AS hash_type
FROM   pg_authid WHERE rolcanlogin;

-- Reset each one, which re-hashes with the new algorithm
ALTER ROLE xxxx_app PASSWORD 'strong-password';

-- Only then switch pg_hba.conf lines to scram-sha-256 and reload.
```

ORDER OF OPERATIONS

Change `password_encryption` and reset every password **before** changing `pg_hba.conf`. Reversing the order locks out every account whose stored hash is still md5.

8.6 A user cannot connect to PostgreSQL. How will you troubleshoot it?

Answer. Work outward from the network to the privilege. The error message tells you which layer failed — read it before doing anything.

ERROR MESSAGE	LAYER THAT FAILED
could not connect to server: Connection refused	Server down, wrong port, or <code>listen_addresses</code>
timeout expired / no response	Firewall or security group
no <code>pg_hba.conf</code> entry for host ...	<code>pg_hba.conf</code> — no matching rule
password authentication failed for user	Wrong password, or hash/method mismatch
role "x" does not exist	Role missing or misspelled
database "x" does not exist	Wrong database name
permission denied for database	No <code>CONNECT</code> privilege
too many connections for role	<code>CONNECTION LIMIT</code> on the role
sorry, too many clients already	<code>max_connections</code> reached cluster-wide
fe_sendauth: no password supplied	Client sent an empty password — set <code>PGPASSWORD</code> or use a password file

```
# 1. Network reachability
telnet db.host 5432 || nc -zv db.host 5432

# 2. Is the server listening on the right interface?
psql -c "SHOW listen_addresses;" # '*' or the specific IP, not just 'localhost'
psql -c "SHOW port;"

# 3. Does a pg_hba rule match?
psql -c "SELECT line_number, type, database, user_name, address, auth_method, error
        FROM pg_hba_file_rules ORDER BY line_number;"

# 4. Does the role exist, can it log in, has the password expired?
psql -c "SELECT rolname, rolcanlogin, rolvaliduntil, rolconndefaults
        FROM pg_roles WHERE rolname = 'xxxx_app';"

# 5. Connection headroom
psql -c "SELECT count(*) AS used, current_setting('max_connections') AS max
        FROM pg_stat_activity;"

# 6. The server log is authoritative
tail -100 /var/lib/pgsql/17/data/log/postgresql-*.log
```

9 PostgreSQL Upgrades

Minor upgrades are routine. Major upgrades are a project.

9.1 What is the difference between a minor and a major upgrade?

	MINOR (17.2 → 17.4)	MAJOR (16.X → 17.X)
What changes	Bug and security fixes only	New features; on-disk format may change
Data directory	Unchanged — fully compatible	Must be converted or reloaded
Procedure	Install binaries, restart	<code>pg_upgrade</code> , dump/restore, or logical replication
Downtime	Seconds	Minutes to hours, depending on method
Rollback	Reinstall the old binaries	Restore from backup, or fail back to the old cluster
Extensions	Unaffected	Must be rebuilt for the new major version

```
# Minor upgrade - this really is the whole procedure
systemctl stop postgresql
dnf update postgresql17-server postgresql17
systemctl start postgresql
psql -c "SELECT version();"

```

APPLY MINOR RELEASES

Minor versions are cumulative bug and security fixes with no format change. Deferring them accumulates known defects — including data-corruption fixes — for no benefit. Adopt a quarterly cadence.

9.2 How do you perform a PostgreSQL major-version upgrade?

Method 1 — `pg_upgrade --link` (fastest, minutes even on multi-TB)

```
# 1. Install the new major version alongside the old
dnf install -y postgresql17-server postgresql17-contrib
/usr/pgsql-17/bin/initdb -D /var/lib/pgsql/17/data

# 2. CHECK FIRST - this changes nothing and reports every incompatibility
/usr/pgsql-17/bin/pg_upgrade \
  --old-datadir=/var/lib/pgsql/16/data --new-datadir=/var/lib/pgsql/17/data \
  --old-bindir=/usr/pgsql-16/bin      --new-bindir=/usr/pgsql-17/bin \
  --check

# 3. Stop both clusters, then upgrade
systemctl stop postgresql-16 postgresql-17
/usr/pgsql-17/bin/pg_upgrade \
  --old-datadir=/var/lib/pgsql/16/data --new-datadir=/var/lib/pgsql/17/data \
  --old-bindir=/usr/pgsql-16/bin      --new-bindir=/usr/pgsql-17/bin \
  --link --jobs=8
# --link uses hard links: near-instant, but the OLD CLUSTER IS THEN UNUSABLE.
# Omit --link to copy instead: slower, but the old cluster remains a rollback path.

# 4. Start, and REBUILD STATISTICS - pg_upgrade does not carry them over
systemctl start postgresql-17
/usr/pgsql-17/bin/vacuumdb --all --analyze-in-stages --jobs=8

# 5. Only after verification
./delete_old_cluster.sh

```

Method 2 — logical replication (near-zero downtime)

```
-- On the OLD cluster (publisher)
ALTER SYSTEM SET wal_level = 'logical'; -- restart required
CREATE PUBLICATION xxxx_pub FOR ALL TABLES;

-- On the NEW cluster (subscriber), schema already created
CREATE SUBSCRIPTION xxxx_sub
  CONNECTION 'host=old.host dbname=xxxx user=replicator password=xxx'
  PUBLICATION xxxx_pub;

-- Watch until caught up, then cut over during a brief write freeze
SELECT * FROM pg_stat_subscription;
-- Reset every sequence on the new cluster - logical replication does NOT copy them.
```

Method 3 — dump and restore

Simplest and slowest. Acceptable below roughly 100 GB, or when you also want to defragment and re-plan the schema.

9.3 What is the difference between pg_upgrade and logical replication for upgrades?

	PG_UPGRADE	LOGICAL REPLICATION
Downtime	Minutes (with <code>--link</code>)	Seconds — only the cutover
Complexity	Low	High
Rollback	Hard after <code>--link</code>	Easy — the old cluster is untouched and still current
Extra hardware	None (same server)	A second server
Statistics	Lost — must re- <code>ANALYZE</code>	Rebuilt naturally on the new cluster
Sequences	Carried over	NOT replicated — must be reset by hand
Tables without a PK	Fine	Need <code>REPLICA IDENTITY FULL</code>
DDL during migration	N/A	Not replicated — freeze schema changes
Also changes OS/hardware	No	Yes — you can move platform at the same time

CHOOSING

pg_upgrade when a short maintenance window is acceptable — which covers most systems. **Logical replication** when downtime must be seconds, or when you are moving to new hardware, a new OS, or a new cloud at the same time. The extra complexity buys exactly those two things.

9.4 What checks would you perform before upgrading production?

#	CHECK	COMMAND OR ACTION
1	Full backup, verified restorable	<code>pgbackrest backup</code> then an actual test restore
2	Read the release notes for every intervening major version	Incompatibilities are listed there, nowhere else
3	<code>pg_upgrade --check</code>	Reports incompatibilities without changing anything
4	Extension availability in the new version	<code>SELECT * FROM pg_extension;</code> then confirm each is packaged

#	CHECK	COMMAND OR ACTION
5	Rehearse on a copy of production	Not on a small dev database. Time it.
6	Application compatibility	Driver versions, deprecated syntax, changed defaults
7	Disk space	Copy mode needs 2×; <code>--link</code> needs little
8	Capture the current plans for critical queries	So you can compare afterwards
9	Replication topology	Standbys must be rebuilt or upgraded in the right order
10	Written rollback plan with a decision deadline	Rehearsed, not theoretical

```
# Inventory before you touch anything
psql -c "SELECT version();"
psql -c "SELECT * FROM pg_extension;"
psql -c "SELECT name, setting FROM pg_settings WHERE source='configuration file';"
psql -c "SELECT datname, pg_size_pretty(pg_database_size(datname)) FROM pg_database;"
psql -c "SELECT * FROM pg_stat_replication;"
```

9.5 How would you plan rollback if the upgrade fails?

METHOD USED	ROLLBACK	TIME
<code>pg_upgrade copy</code> mode	Start the old cluster — it is untouched	Minutes
<code>pg_upgrade --link</code>	No fallback. Restore from backup	Hours
Dump and restore	Old cluster still running	Minutes
Logical replication	Repoint applications back; optionally reverse the replication	Seconds

```
# Rollback checklist to write down BEFORE the window opens
# 1. Decision deadline - "if not verified by 03:00, we roll back". Non-negotiable.
# 2. Who makes the call, and who executes it
# 3. Exact commands, already written and tested
# 4. How applications are repointed (connection string, VIP, DNS TTL)
# 5. How to verify success after rollback - the same tests as after the upgrade
# 6. Communication plan
```

THE TRADE-OFF IN `--LINK`

`--link` makes a multi-terabyte upgrade take minutes instead of hours — but it hard-links the data files, so the **old cluster becomes unusable the moment the new one starts**. Your only rollback is a full restore. Use copy mode when the window allows it; use `--link` only when the database is too large for copy mode and you have accepted, in writing, that rollback means a restore.

THE SAFEST PATTERN

Take a fresh verified backup, upgrade with **copy** mode, keep the old cluster stopped but intact for a full business cycle, and only run `delete_old_cluster.sh` after a week of clean production running.

10 Ten Production Scenarios

The scenarios that come up most often, and the shape of answer an interviewer is listening for.

SC-01

The database is very slow

Interviewer expects: *pg_stat_activity, blocking, CPU, I/O, slow queries — in that order*

```
SELECT pid, state, wait_event_type, wait_event, now()-query_start AS runtime, left(query,80)
FROM pg_stat_activity WHERE state <> 'idle' ORDER BY query_start;
```

```
SELECT pid, pg_blocking_pids(pid), left(query,80) FROM pg_stat_activity
WHERE cardinality(pg_blocking_pids(pid)) > 0;
```

```
SELECT round(total_exec_time::numeric) AS ms, calls, left(query,80)
FROM pg_stat_statements ORDER BY total_exec_time DESC LIMIT 10;
```

Say this: “First I check what is running and what it is waiting on. `wait_event_type` tells me immediately whether this is locking, I/O or CPU, and that decides everything I do next. I do not change any parameter until I know which.”

SC-02

CPU is at 100%

Interviewer expects: *Identify the expensive queries, not the busy processes*

```
SELECT pid, now()-query_start AS runtime, left(query,100) FROM pg_stat_activity
WHERE state='active' AND wait_event IS NULL ORDER BY query_start;
```

```
top -c -o %CPU      # backend PID matches pg_stat_activity.pid
```

Say this: “An active backend with a NULL `wait_event` is on the CPU right now. I map those PIDs straight to `top`. The cause is usually a missing index or a plan that flipped after statistics went stale — so I check `EXPLAIN` and `last_analyze` before I consider hardware.”

SC-03

Disk is 95% full

Interviewer expects: *WAL, replication slots, bloat, logs — check slots first*

```
SELECT slot_name, active, wal_status,
       pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)) AS retained
FROM pg_replication_slots ORDER BY 4 DESC;
```

```
SELECT failed_count, last_failed_wal FROM pg_stat_archiver;
du -sh $PGDATA/pg_wal $PGDATA/log $PGDATA/base/pgsql_tmp
```

Say this: “An inactive replication slot is the single most common cause. I check that first, then a failing `archive_command`, then bloat. I never delete files from `pg_wal` by hand — I fix the cause and let PostgreSQL recycle them.”

SC-04

The primary is down

Interviewer expects: Diagnose, fence, promote, repoint, rebuild

```
ssh primary 'systemctl status postgresql; tail -100 $PGDATA/log/*.log; df -h'
psql -h standby -c "SELECT pg_last_wal_replay_lsn(), pg_last_xact_replay_timestamp();"
ssh primary 'systemctl stop postgresql && systemctl disable postgresql' # FENCE
psql -h standby -c "SELECT pg_promote(wait => true);"
```

Say this: “I check whether it is genuinely down before promoting — often it just needs starting. If I must fail over, I fence the old primary **before** promoting, because promoting while it can still accept writes is how split brain happens. I record the standby's replay position first so I can state the actual data loss.”

SC-05

The standby has high lag

Interviewer expects: Distinguish transport lag from apply lag

```
SELECT pg_last_wal_receive_lsn() AS received, pg_last_wal_replay_lsn() AS replayed,
       pg_wal_lsn_diff(pg_last_wal_receive_lsn(), pg_last_wal_replay_lsn()) AS to_apply;

SELECT pid, wait_event_type, wait_event, now()-query_start, left(query,80)
FROM pg_stat_activity WHERE state='active';
```

Say this: “If received is behind the primary it is a network problem. If received is far ahead of replayed, the standby cannot apply fast enough — usually a long query on the standby conflicting with recovery. Then it is a choice between `max_standby_streaming_delay` and `hot_standby_feedback`, and I would say which side pays the cost.”

SC-06

A query suddenly became slow

Interviewer expects: Plan, statistics, indexes — in that order

```
SELECT relname, n_live_tup, n_dead_tup, last_analyze, last_autoanalyze
FROM pg_stat_user_tables WHERE relname='xxxx_orders';

ANALYZE xxxx_orders;
EXPLAIN (ANALYZE, BUFFERS) SELECT ...;
SET plan_cache_mode = force_custom_plan; -- test for a generic-plan flip
```

Say this: “The query did not change, so something around it did. I compare estimated against actual rows in the plan — a large gap means stale statistics. If statistics are current I check for a generic plan from a prepared statement, then bloat, then whether it is actually blocked rather than slow.”

SC-07

Table size keeps growing

Interviewer expects: Dead tuples, blocked VACUUM, bloat

```
SELECT relname, n_live_tup, n_dead_tup, last_autovacuum
FROM pg_stat_user_tables ORDER BY n_dead_tup DESC LIMIT 10;

SELECT max(age(backend_xmin)) FROM pg_stat_activity;
SELECT slot_name, active, age(xmin) FROM pg_replication_slots;
```

Say this: “If `last_autovacuum` is recent but dead tuples keep climbing, VACUUM is running and freeing nothing — something is holding the xmin horizon. That is a long transaction, an abandoned replication slot, or an orphaned prepared transaction. Repacking without finding that just delays the problem.”

SC-08

A database was accidentally dropped

Interviewer expects: PITR to just before the event

```
cp -a $PGDATA/pg_wal /safe/pg_wal_rescue      # preserve unarchived WAL first
# restore base backup to a SEPARATE directory, then:
#   restore_command      = 'cp /wal_archive/%f %p'
#   recovery_target_time = '2026-09-09 13:59:59+05:30'
#   recovery_target_action = 'pause'
#   port = 5433
touch $NEWDATA/recovery.signal && pg_ctl -D $NEWDATA start
```

Say this: “I restore to a separate directory on a separate port, verify the data is there while recovery is paused, and only then either extract what was lost or promote. Restoring over production gives you one attempt and no way back.”

SC-09

A user cannot connect

Interviewer expects: Read the error message — it names the layer

```
nc -zv db.host 5432
psql -c "SHOW listen_addresses;"
psql -c "SELECT line_number, type, database, user_name, address, auth_method, error
        FROM pg_hba_file_rules ORDER BY line_number;"
psql -c "SELECT rolname, rolcanlogin, rolvaliduntil FROM pg_roles WHERE rolname='xxxx_app';"
tail -100 $PGDATA/log/postgresql-*.log
```

Say this: “The error text tells me the layer. *No `pg_hba.conf` entry* is authentication rules; *password authentication failed* is credentials or a hash-method mismatch; *connection refused* is the network or `listen_addresses`. And in `pg_hba.conf` the first matching line wins, so a broad rule above a specific one is a very common cause.”

SC-10

Production major-version upgrade

Interviewer expects: Verified backup, --check, rehearse, rollback plan

```
pgbackrest --stanza=xxxx backup && pgbackrest --stanza=xxxx verify
/usr/pgsql-17/bin/pg_upgrade ... --check
/usr/pgsql-17/bin/pg_upgrade ... --link --jobs=8
vacuumdb --all --analyze-in-stages --jobs=8      # MANDATORY afterwards
```

Say this: “I take and verify a backup, run `pg_upgrade --check`, rehearse on a copy of production and time it, then execute with a written rollback plan and a decision deadline. Afterwards `ANALYZE` is mandatory — `pg_upgrade` does not carry statistics across, and skipping it is the usual reason people think the new version is slower.”

11 Command Cheat Sheet

Twenty queries that cover the large majority of day-to-day diagnosis. Worth committing to memory.

Who is running what

```
SELECT pid, state, wait_event_type, wait_event, now()-query_start AS runtime,
       left(query,80)
FROM pg_stat_activity WHERE state <> 'idle' ORDER BY query_start;
```

Who is blocked, and by whom

```
SELECT pid, pg_blocking_pids(pid) AS blocked_by, left(query,80)
FROM pg_stat_activity WHERE cardinality(pg_blocking_pids(pid)) > 0;
```

Cancel / kill a session

```
SELECT pg_cancel_backend(PID);      -- cancels the statement
SELECT pg_terminate_backend(PID);  -- closes the connection
```

Top 10 queries by total time

```
SELECT round(total_exec_time::numeric) AS ms, calls,
       round(mean_exec_time::numeric,2) AS mean_ms, left(query,80)
FROM pg_stat_statements ORDER BY total_exec_time DESC LIMIT 10;
```

Dead tuples / vacuum status

```
SELECT relname, n_live_tup, n_dead_tup, last_autovacuum, last_autoanalyze
FROM pg_stat_user_tables ORDER BY n_dead_tup DESC LIMIT 20;
```

Vacuum in progress

```
SELECT pid, relid::regclass, phase,
       round(100.0*heap_blks_scanned/NULLIF(heap_blks_total,0),1) AS pct
FROM pg_stat_progress_vacuum;
```

Wraparound risk

```
SELECT datname, age(datfrozenxid),
       2000000000 - age(datfrozenxid) AS xids_left
FROM pg_database ORDER BY 2 DESC;
```

Replication status (primary)

```
SELECT application_name, state, sync_state,
       pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), replay_lsn)) AS behind
FROM pg_stat_replication;
```

Replication lag (standby)

```
SELECT CASE WHEN pg_last_wal_receive_lsn() = pg_last_wal_replay_lsn() THEN 0
            ELSE EXTRACT(epoch FROM now()-pg_last_xact_replay_timestamp())
            END AS lag_seconds;
```

Replication slots

```
SELECT slot_name, active, wal_status,  
       pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)) AS retained  
FROM pg_replication_slots ORDER BY 4 DESC;
```

Archiver health

```
SELECT archived_count, failed_count, last_archived_time,  
       last_failed_wal, last_failed_time FROM pg_stat_archiver;
```

Largest tables

```
SELECT relname, pg_size_pretty(pg_total_relation_size(relid)) AS total  
FROM pg_catalog.pg_statio_user_tables  
ORDER BY pg_total_relation_size(relid) DESC LIMIT 20;
```

Largest databases

```
SELECT datname, pg_size_pretty(pg_database_size(datname)) AS size  
FROM pg_database ORDER BY pg_database_size(datname) DESC;
```

Unused indexes

```
SELECT relname, indexrelname,  
       pg_size_pretty(pg_relation_size(indexrelid)) AS size  
FROM pg_stat_user_indexes WHERE idx_scan = 0  
ORDER BY pg_relation_size(indexrelid) DESC;
```

Cache hit ratio

```
SELECT sum(heap_blks_hit)::numeric  
       / NULLIF(sum(heap_blks_hit)+sum(heap_blks_read),0) AS hit_ratio  
FROM pg_statio_user_tables;
```

Connections in use

```
SELECT count(*) AS used, current_setting('max_connections') AS max  
FROM pg_stat_activity;
```

Non-default settings

```
SELECT name, setting, source FROM pg_settings  
WHERE source NOT IN ('default','override') ORDER BY name;
```

Active pg_hba rules

```
SELECT line_number, type, database, user_name, address, auth_method, error  
FROM pg_hba_file_rules ORDER BY line_number;
```

Oldest open transaction

```
SELECT pid, username, now()-xact_start AS age, state, left(query,80)  
FROM pg_stat_activity WHERE xact_start IS NOT NULL ORDER BY xact_start LIMIT 5;
```

Temp file spills (work_mem too small)

```
SELECT datname, temp_files, pg_size_pretty(temp_bytes)
FROM pg_stat_database WHERE temp_files > 0 ORDER BY temp_bytes DESC;
```

12 Suggested Study Order

Each topic builds on the one before it. Studied in this order, the later material needs far less memorising.

ORDER	TOPIC	CORE CONCEPTS	WHY HERE
1	Performance	pg_stat_activity, EXPLAIN, pg_stat_statements, scan types	Every incident starts here, and every interview does too.
2	Locks and blocking	pg_blocking_pids, lock modes, cancel vs terminate	The most common real incident, and the fastest to diagnose once you know the query.
3	VACUUM and autovacuum	MVCC, bloat, xmin horizon, wraparound	The topic that separates a DBA from a user. Most 'mystery' problems end up here.
4	Backup and PITR	pg_dump vs pg_basebackup, WAL archive, recovery_target_time	The only area where being wrong is unrecoverable.
5	Replication	streaming, slots, sync vs async, lag	Prerequisite for HA, and for understanding WAL retention.
6	HA and failover	fencing, split brain, Patroni, pg_rewind	Builds directly on replication.
7	WAL	segments, archiving, recycling	Underpins backup, replication and most disk incidents.
8	Disk and bloat	slots, archiver, pgstattuple, pg_repack	Frequent pages; nearly always one of five causes.
9	Security	roles, GRANT, pg_hba.conf, scram-sha-256	Less frequent, but a wrong answer is memorable.
10	Upgrades	minor vs major, pg_upgrade, logical replication, rollback	Project work rather than incident work.

THE THREE ANSWERS THAT CARRY THE MOST WEIGHT

1. "A backup that has never been restored is not a backup." — monthly rehearsals, measured RTO.
2. "I fence before I promote." — you understand split brain.
3. "VACUUM is running but freeing nothing, so something is holding the xmin horizon." — you understand MVCC rather than just the command.

WHAT SEPARATES A STRONG CANDIDATE

Not knowing more commands. It is naming the **diagnostic order** before reaching for any command, saying which observation would change your mind, and being explicit about trade-offs — `hot_standby_feedback` moves cost from the standby to the primary; `--link` trades rollback for speed; `max_slot_wal_keep_size` trades a standby for the primary's disk. State the trade, then state your choice.